

Hook errors in quantum error correction

Carson Chan

Msci. Theoretical Physics

Supervisor: Dan Browne

Word Count:7480

Abstract

Hook errors — correlated multi-qubit errors arising from the propagation of a single ancilla fault through entangling gates during stabilizer measurements — can cause logical failures with fewer physical errors than the code distance would suggest. The ordering of CNOT gates within stabilizer measurement circuits determines how hook errors propagate, and optimising this ordering offers a practical route to reducing logical error rates without increasing qubit overhead. This work investigates gate order optimisation for the unrotated surface code and the 2D triangular colour code through numerical simulation using Stim.

For the unrotated surface code, all CNOT gate orderings are simulated under both a simple noise model isolating hook error effects and a realistic circuit-level noise model with varying idle error rates. For the range of idle error rate simulated, the interleaving-compatible gate order is found to be consistently among the worst performers for hook error suppression, implying that non-interleaving schedules can achieve lower logical error rates on platforms with sufficiently low idle error rates such as ion trap quantum computers. Performance rankings are observed to change with varying idle error rates, demonstrating a non-trivial gate-order-dependent interaction between hook errors and idle errors. Consistent behaviour is observed in the 2D triangular colour code, where the optimal gate order schedule is identified and schedules grouped into statistically distinguishable performance clusters, suggesting the interaction could be a general feature of stabilizer circuits. These results motivate detailed theoretical investigation of hook error interaction mechanisms and hardware-specific studies to determine the conditions under which optimised schedules provide a practical advantage.

Contents

Introduction	5
1 Research Aims of the Project.....	7
2 Theory	8
2.1 Quantum Circuits.....	8
2.1.1 Controlled Operation	9
2.1.2 Circuit Identities	9
2.2 Quantum Error Correction	11
2.2.1 Surface Codes.....	12
2.2.2 Colour Codes	14
2.2.3 Hook Errors	16
2.3 Threshold theorem for Fault Tolerant Quantum Computation	17
2.4 Noise Models.....	18
3 Methods	19
3.1 Simulation Procedures	20
3.2 Gate Order in Stabilizer Measurements.....	21
3.2.1 Stabilizer Gate Orders in the Unrotated Surface Code.....	21
3.2.2 Stabilizer Gate Orders in the 2D Triangular Colour Code	22
3.3 Statistical Distinguishability (2D Triangular Colour Code)	24
4 Results	25
4.1 Surface Codes Results	25
4.1.1 $d=3$ Unrotated Surface Code Under Simple Noise Model	25
4.1.2 Unrotated Surface Code Under Realistic Noise Model	27
4.2 Colour Codes Results.....	33
5 Discussion.....	39
6 Summary and Conclusions	41

References	42
Appendix	44
ABC Theorem	44
Universal Quantum Computation	44
Full Results for The Triangular Colour Code	44

Introduction

Quantum computing is a computational paradigm that utilises quantum phenomena (e.g. superposition, entanglement) to perform certain tasks more efficiently than classical computers, which is often referred to as “quantum advantage”. For example, Shor’s algorithm can factor large integers exponentially faster than known classical algorithms, and quantum simulation [14] offers the potential to model complex quantum systems beyond classical reach. These capabilities have driven extensive research into scalable quantum hardware and algorithms.

A central challenge in building practical quantum computers is the fragility of quantum information. Physical qubits suffer from decoherence and gate errors due to interactions with their environment and imperfect control operations. Unlike classical bits, qubits are susceptible to both bit-flip and phase-flip errors, and even small error rates rapidly degrade quantum computations if left uncorrected.

Quantum error correction (QEC) provides a route to practical quantum computing by encoding logical qubits into ensembles of physical qubits and repeatedly detecting and correcting errors without collapsing the quantum information. Under the threshold theorem, if the physical error rates are below a critical threshold, logical error rates can be suppressed exponentially by increasing the code distance, where a distance- d code can correct up to $(d - 1)/2$ independent errors (formally, the minimum number of physical qubit errors required to cause an undetectable logical error), at the cost of a larger number of physical qubits. This behaviour has recently been demonstrated in experiments with surface codes: for example, a distance-7 surface code implemented on a 105-qubit superconducting processor achieved a logical error rate per cycle of approximately 0.143 %, and the lifetime of the logical qubit exceeded that of all the constituent physical qubits by a factor of ~ 2.4 , showing clear below-threshold operation and demonstrating suppression of logical errors with increasing code size [13].

While increasing code distance and physical qubit count reduces logical errors, it also increases the circuit complexity and overhead. In addition to code scaling, researchers have explored circuit-level optimisation strategies to reduce logical error rates [15] without simply adding more qubits. These strategies include optimising the order and pattern of entangling gates in stabilizer measurement circuits, careful scheduling to avoid correlated error propagation (e.g. *hook errors*, discussed in section 2.2.3), and tailoring syndrome extraction patterns to minimise harmful error combinations that degrade logical performance. Optimised stabilizer circuits can lower effective logical error rates or raise

the operational threshold under given noise models, making error correction effective in noisier environments or with fewer resources.

Optimising circuit patterns for QEC can make achieving a target logical error rate more *resource-efficient* or *scalable*: it may reduce the number of physical qubits needed for a given logical performance, relax hardware fidelity requirements, or improve decoder performance by making syndrome data more robust to correlated error events. However, such optimisation can increase timesteps—for example, in the surface code, by using a gate order not compatible with interleaving, doubling the number of discrete steps required for the CNOT gates (controlled-NOT) in the stabilizer extraction schedule. While this slows down computation for a fixed number of measurement rounds and increases exposure to idle decoherence errors the trade-off may be more favourable on hardware platforms with characteristically lower idle error rates, such as ion trap quantum computers compared to superconducting ones, though whether it is truly advantageous for any specific device requires a device-specific noise analysis beyond the scope of this work.

1 Research Aims of the Project

Building on the motivation outlined above, this project investigates gate order optimisation for two topological codes: the unrotated surface code and the 2D triangular colour code.

The specific aims are as follows:

1. To investigate circuit pattern optimisation strategies for the unrotated surface code and the 2D triangular colour code that reduce logical error rates by minimising the impact of hook errors. This involves systematically simulating all distinct CNOT gate orderings within the stabilizer measurement cycles of each code and identifying which schedules best suppress hook error propagation.
2. To analyse the impact of gate order optimisation on logical error suppression and threshold behaviour, and to determine under what conditions the trade-off associated with the optimisation strategy — in particular the potential incompatibility with interleaved stabilizer scheduling and the resulting increase in circuit depth — remains beneficial. The findings are intended to inform the design of scalable fault-tolerant quantum computing architectures, including identifying hardware platforms, such as those with characteristically low idle error rates, where the optimised schedules are most likely to provide a practical advantage.
3. To motivate detailed theoretical study of the interaction mechanisms between hook errors and other error channels, by investigating how the presence of additional error mechanisms — in particular idle errors at varying rates — affects the relative performance ranking of gate orders in both the unrotated surface code and the 2D triangular colour code. A change in performance ranking under a more realistic noise model would constitute evidence that hook errors interact non-trivially with other noise channels and would provide empirical grounding for future analytical work aimed at characterising these interactions."

2 Theory

2.1 Quantum Circuits

Operations on qubit states are described by unitary matrices such that the norm is preserved, i.e. the total probability remains to be 1. Time proceeds from left to right in a circuit diagram. Wires represent qubits.

The state of a qubit (quantum bit) can be represented as a linear combination of the states $|0\rangle, |1\rangle$, each corresponding to the classical bit states 0 and 1 :

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

Where α, β are complex numbers and $|\alpha|^2 + |\beta|^2 = 1$. The states $|0\rangle, |1\rangle$ form an orthonormal basis and is known as the *computational basis*(Z basis).

Single-qubit operations are described by 2x2 unitary matrices. Some common single qubit gates include the Pauli X,Y,Z matrices, Hadamard gate $H = \frac{1}{\sqrt{2}}(X + Z)$, and the T $\frac{\pi}{8}$ gate.

Gate	Symbol	Matrix Representation	Circuit Diagram
Pauli-X	X	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	
Pauli-Y	Y	$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$	
Pauli-Z	Z	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	
Hadamard	H	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$	
Phase	S	$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$	
T gate	T	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$	

Figure 1 Some common single qubit gates.

2.1.1 Controlled Operation

A controlled-U operation $C^n(U)$ with n control qubits is one that applies the operation U to the target if and only if all the control qubits are in state $|1\rangle$. It is defined as:

$$C^n(U)|x_1, x_2, \dots, x_n\rangle|\psi\rangle = |x_1, x_2, \dots, x_n\rangle U^{x_1 x_2 \dots x_n} |\psi\rangle$$

The case $n = 1$ and $U = X$ is an important example and is known as the controlled-NOT (CNOT or CX) gate, which has the effect of changing $|0\rangle$ to $|1\rangle$ and $|1\rangle$ to $|0\rangle$ in the target state when the control qubit is in $|1\rangle$, i.e. $CNOT|a, b\rangle = |a, b \oplus a\rangle$ where $\oplus$ is addition modulo 2. The CNOT gate can be represented in matrix form as follows:

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

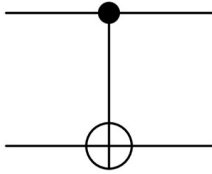


Figure 2 circuit diagram of the CNOT gate.

2.1.2 Circuit Identities

Quantum circuits can be simplified using circuit identities. For example,

$$HXH = Z; \quad HYH = -Y; \quad HZH = X.$$

More identities can be found in [10].

Some important circuit identities are those of the CNOT gate. We show by applying to a state $|a\rangle|b\rangle = |a, b\rangle$ that $(X \otimes I)CNOT = CNOT(X \otimes X)$:

$$(X \otimes I)CNOT|a, b\rangle = (X \otimes I)|a, b \oplus a\rangle = |a \oplus 1, b \oplus a\rangle$$

$$CNOT(X \otimes X)|a, b\rangle = CNOT|a \oplus 1, b \oplus 1\rangle = |a \oplus 1, b \oplus 1 \oplus (a \oplus 1)\rangle = |a \oplus 1, b \oplus a\rangle.$$

Similarly, $(I \otimes Z)CNOT = CNOT(Z \otimes Z)$:

$$(I \otimes Z)CNOT|a, b\rangle = (I \otimes Z)|a, b \oplus a\rangle = (-1)^{b \oplus a}|a, b \oplus a\rangle$$

$$CNOT(Z \otimes Z)|a, b\rangle = CNOT(-1)^{a+b}|a, b\rangle = (-1)^{a+b}|a, b \oplus a\rangle = (-1)^{b \oplus a}|a, b \oplus a\rangle,$$

while $(I \otimes X)$ and $(Z \otimes I)$ commute through the gate, i.e. $(I \otimes X)CNOT = CNOT(I \otimes X)$ and $(Z \otimes I)CNOT = CNOT(Z \otimes I)$:

$$(I \otimes X)CNOT|a, b\rangle = (I \otimes X)|a, b \oplus a\rangle = |a, b \oplus a \oplus 1\rangle$$

$$CNOT(I \otimes X)|a, b\rangle = CNOT|a, b \oplus 1\rangle = |a, b \oplus 1 \oplus a\rangle = |a, b \oplus a \oplus 1\rangle;$$

$$(Z \otimes I)CNOT|a, b\rangle = (Z \otimes I)|a, b \oplus a\rangle = (-1)^a|a, b \oplus a\rangle$$

$$CNOT(Z \otimes I)|a, b\rangle = (-1)^aCNOT|a, b\rangle = (-1)^a|a, b \oplus a\rangle.$$

These identities describe how Pauli errors propagate through entangling gates and will be important when analysing error behaviour in multi-qubit circuits.

2.2 Quantum Error Correction

As a result of the *no-cloning theorem* [1][6], quantum information cannot be copied, therefore redundancy is introduced in a fundamentally different way from classical systems. The qubit is encoded into an entangled state of multiple qubits known as *logical qubits*. For example, if 3 qubits are used then $|0\rangle \rightarrow |0_L\rangle = |000\rangle$.

Despite the possible errors on a qubit state is continuous, it can be shown that it is sufficient to only correct a finite set of errors to protect against any arbitrary errors. In particular, the set $\{I_2, X, Y, Z\}$ forms a basis for 2×2 matrices, thus correcting X, Y, Z is sufficient for any 1 qubit errors. In general, a code that corrects A and B also correct any linear combination of A and B [1]. In other words, errors can be discretised.

In QEC, error detection is done through syndrome measurements, which shows whether an error has occurred and reveals nothing about the original encoded state, so that the original state is not disturbed. The operators for error detection are called *stabilizers*. The valid (error free) code states are simultaneous $+1$ eigenstates of all stabilizers.

Early error correcting codes which protect against arbitrary noise include the *Shor code* [3], which is a combination of phase and bit flip codes. A more recent and ancilla-efficient error correcting code is the *flag fault-tolerant error-correction scheme* [4], where syndrome is extracted into an ancilla qubit, on which a single error can propagate into multiple errors (“hook errors”). It is therefore protected by a “flag gadget”, which consists of two CNOT gates and a measurement. Correlated errors can be detected from the measurement outcome of the flag ancilla. A code with distance d can correct at most $(d - 1)/2$ independent errors.

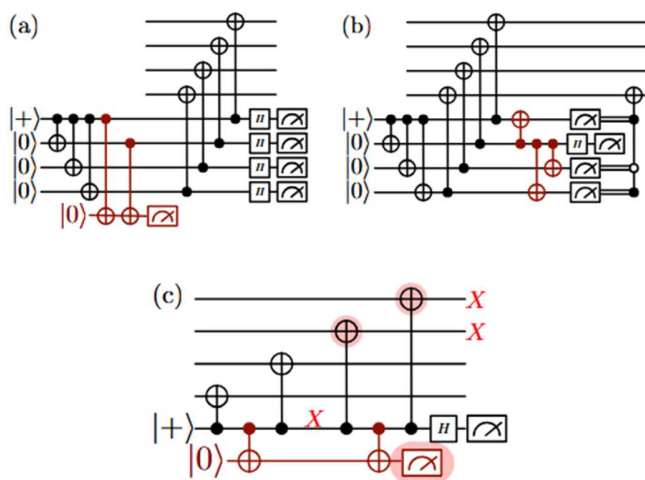


Figure 3 (a) Shor's method (b) DiVincenzo-Aliferis decoding method (c) Flag method [4]. These are some examples of error correcting codes, which has additional qubits in the circuit to detect errors.

2.2.1 Surface Codes

A promising way to achieve large scale quantum computing is by using *surface codes* [7] due to its relatively high tolerance to errors. It is implemented on a 2D array of physical qubits; each are either data or measurement(ancilla) qubits. There are 2 types of measurement qubits, measuring Z and X stabilizers and are called “Z-syndrome” and “X-syndrome” qubits, two of each are coupled to every data qubit. Similarly, each syndrome qubit couples to four neighbouring data qubits. Each measurement qubits are initialised in the ground state $|g\rangle$. Z-syndrome qubits comprises 4 CNOT gates, with the neighbouring data qubits act as the control, follows with measurement yielding eigenstate of the 4-qubit stabilizer $\hat{Z}_a\hat{Z}_b\hat{Z}_c\hat{Z}_d$ (“vertex stabilizers”), whereas the X-syndrome qubits act as the control, targeting the 4 neighbouring data qubits, with Hadamard applied before and after the sequence of CNOTs, which yields eigenstates of $\hat{X}_a\hat{X}_b\hat{X}_c\hat{X}_d$ (“plaquette stabilizers”). After measurement, the state $|\psi\rangle$ (“quiescent state”) of all data qubits are simultaneously eigenstates of $\hat{Z}_a\hat{Z}_b\hat{Z}_c\hat{Z}_d$ and $\hat{X}_a\hat{X}_b\hat{X}_c\hat{X}_d$. (i.e. $\hat{Z}_a\hat{Z}_b\hat{Z}_c\hat{Z}_d|\psi\rangle = Z_{abcd}|\psi\rangle$, $\hat{X}_a\hat{X}_b\hat{X}_c\hat{X}_d|\psi\rangle = X_{abcd}|\psi\rangle$ with $Z_{abcd} = \pm 1$ and $X_{abcd} = \pm 1$.) The cycle is repeated after measurement. When there is no error, the same state is maintained, yielding the same eigenvalue for the syndrome measurements as the previous cycle, as the stabilizers commute.

The quiescent state is unchanged until disturbed by errors, which will be detected by the change in measurement outcome. For example, a Z (phase-flip) error on the data qubit a:

$$\begin{aligned}\hat{X}_a\hat{X}_b\hat{X}_c\hat{X}_d(\hat{Z}_a|\psi\rangle) &= -\hat{Z}_a(\hat{X}_a\hat{X}_b\hat{X}_c\hat{X}_d|\psi\rangle) \\ &= -X_{abcd}(\hat{Z}_a|\psi\rangle)\end{aligned}$$

The sign change occurs for the two X-syndrome qubits adjacent to data qubit a, allowing localisation of the error. Similarly, an X (bit-flip) error changes the outcome of the two neighbouring Z-syndrome qubits.

In the surface code, the logical error rate p_L roughly scales as

$$p_L \sim \left(\frac{p}{p_{th}}\right)^{(d+1)/2}$$

Where d is the code distance, p is the physical error rate, and p_{th} is the code threshold.

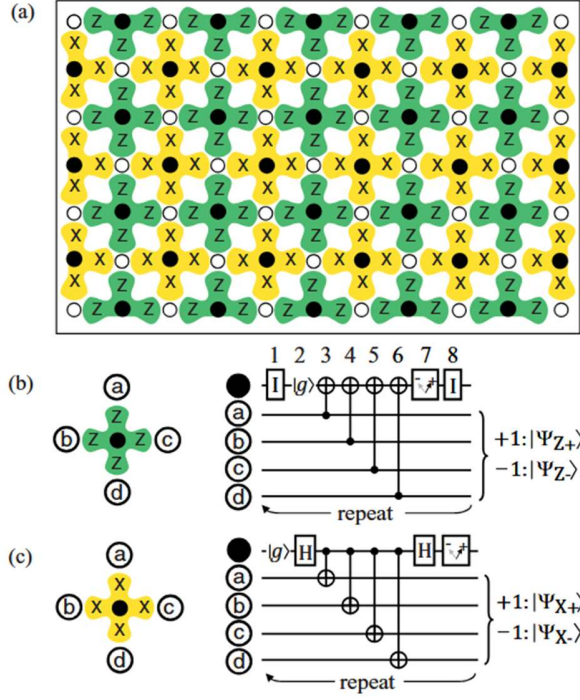


Figure 4 (a) 2D array for the (unrotated) surface code (b) circuit diagram of the Z-syndrome qubit (c) circuit diagram of the X-syndrome qubit [7].

Eigenvalue	$\hat{Z}_a \hat{Z}_b \hat{Z}_c \hat{Z}_d$	$\hat{X}_a \hat{X}_b \hat{X}_c \hat{X}_d$
+1	$ gggg\rangle$	$ ++++\rangle$
	$ ggee\rangle$	$ ++--\rangle$
	$ geeg\rangle$	$ +- - +\rangle$
	$ eegg\rangle$	$ -- ++\rangle$
	$ egge\rangle$	$ - + + -\rangle$
	$ gege\rangle$	$ + - + -\rangle$
	$ egeg\rangle$	$ - + - +\rangle$
	$ eeee\rangle$	$ ----\rangle$
-1	$ ggge\rangle$	$ +++ -\rangle$
	$ ggeg\rangle$	$ ++ - +\rangle$
	$ gegg\rangle$	$ + - ++\rangle$
	$ eggg\rangle$	$ - +++\rangle$
	$ geee\rangle$	$ + ---\rangle$
	$ egee\rangle$	$ - + --\rangle$
	$ eege\rangle$	$ -- + -\rangle$
	$ eeeg\rangle$	$ --- +\rangle$

Figure 5 Eigenvalues for the Z and X stabilizers, where g denotes ground state, e denotes excited state [7].

2.2.2 Colour Codes

Colour codes are defined on lattices that are 3-valent(3 edges at each vertex) and 3-colorable. The convention is to label the 3 colours as red, green and blue. The edges of the lattice can also be assigned colours, where the colour of an edge is the colour of the faces it is not part of. For example, red edges are part of green and blue faces but not red faces [17].

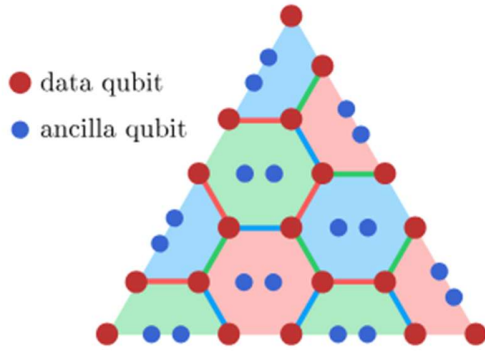


Figure 6 The layout of the physical qubits of a d=5 triangular colour code with both X and Z checks, hence requiring two ancilla qubits per face. Red dots denote data qubits and blue dots denotes ancilla qubits [20].

Colour codes have an important advantage over surface codes that all logical single qubit Clifford gates can be transversely implemented [19], i.e. for some logical single qubit gate U_L , it can be implemented as a tensor product of single qubit gates on each physical qubits:

$$U_L = U_1 \otimes U_2 \dots \otimes U_N = U_1 U_2 \dots U_N$$

hence the implementation of logical gates can be much faster. The code space of the colour codes is the simultaneous +1 eigenspace of all the stabilizer generators.

2D colour codes are CSS stabilizer codes and are specified by the stabilizer generators [18]. The stabilizer generators are local operators acting on the data qubits around each face. One possible geometry of the lattice $\mathcal{L}$ of the 2D colour code is a triangular shape with hexagonal tiling, or equivalently a finite triangular region in a hexagonal lattice, where one logical qubit is encoded. In this case the length of the edge of $\mathcal{L}$ is the code distance , where only odd distances are possible. Qubits are assigned to each vertex in $\mathcal{L}$, X and Z stabilizers are associated with each face $f \in \mathcal{L}$:

$$X_f := \prod_{v \in f} X_v, \quad Z_f := \prod_{v \in f} Z_v$$

Where X_v, Z_v are the Pauli operators on the data qubit v belonging to face f . The stabilizer group $\mathcal{S}$ is generated by the full set of X_f, Z_f of all $f \in \mathcal{L}$.

Syndrome extraction is performed by measuring these stabilizers repeatedly in time. This is achieved by introducing ancilla measurement qubits. Each ancilla qubit is placed at the centre of a plaquette (hexagon) and interacts with the data qubits surrounding the plaquette via a sequence of two-qubit gates. Measuring the ancilla in X basis yields the eigenvalue of the corresponding X -type stabilizer, while measuring in Z basis yields the eigenvalue of the corresponding Z -type stabilizer. A measurement outcome of -1 indicates the presence of an error on one or more of the neighbouring data qubits. The outcomes of stabilizer measurements form the syndrome, which is then used by a classical decoder to infer the most likely error configuration.

The physical layout of the triangular colour code consists of data qubits placed on the vertices of the hexagonal lattice patch, together with syndrome measurement qubits at the centres of the plaquettes. For measurements in both X and Z basis in a single round, two ancilla qubits per face will be needed. During each error correction cycle, stabilizers are measured in a fixed schedule using CNOT gates between ancilla and data qubits, followed by ancilla readout. Repeating these cycles allows both data errors and measurement errors to be detected through changes in syndrome outcomes over time.

2.2.3 Hook Errors

Hook errors arise when a single physical fault during stabilizer measurement spreads to multiple data qubits, “hooking” two errors together [8]. This can be understood using the CNOT propagation identities introduced earlier: an X error on the control qubit propagates to the target, while a Z error propagates in the opposite direction (fig.7). In the surface code for each round of syndrome measurement, the probability of a ZZ hook at a plaquette stabilizer (X^4) or an XX hook at a vertex stabilizer (Z^4) is

$$p_{\text{hook}} = 2p_{\text{CNOT}} + p_s + \text{h. o.}$$

Where h.o. denotes higher order terms, p_s is the probability that a qubit is damaged in a time step, p_{CNOT} the probability that a CNOT gate fails [8]. Correlated X and Z errors does not cause issue as they are dealt with separately.

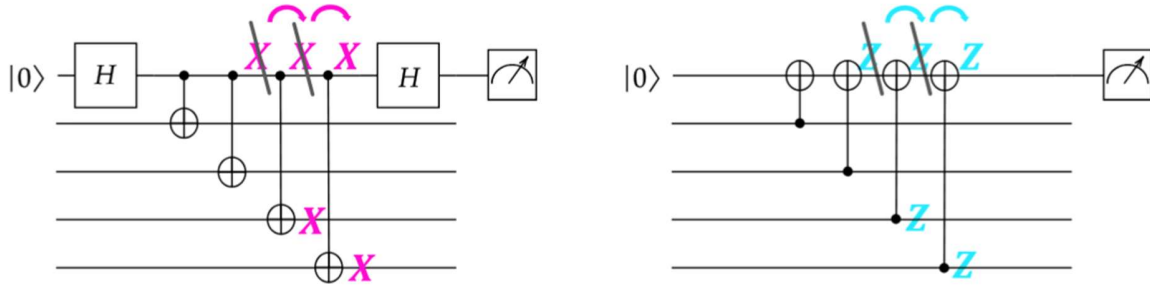


Figure 7 Propagation of Pauli errors in the X and Z stabilizers of the surface code. Copying to 4 data qubits is equivalent to the identity I , copying to 3 is equivalent to a single error up to multiplication by a stabilizer. Therefore, dangerous hooks arise when a single fault propagates to exactly two data qubits, creating a correlated two-qubit error that is not equivalent to a stabilizer and can lead to a logical error if aligned with the code’s logical operators [24].

There are schemes that can mitigate the effects of hook errors such as Shor’s method and the flag method (fig.3), however at a cost of more overhead.

Hook error propagation depends on the syndrome extraction circuit; therefore, an alternative, or complementary way to minimise its effect without introducing additional physical qubits is by changing the circuit pattern. For example, in the surface code, measuring Z stabilizers in a pattern perpendicular to the X -type edges prevents a hook error from spreading Z errors to two different rows of data qubits [11].

A recent paper claimed to have constructed variants of the syndrome extraction circuits for the surface code that prevent hook errors entirely at the cost of larger circuit depth [12].

2.3 Threshold theorem for Fault Tolerant Quantum Computation

A procedure is said to be fault-tolerant if failure in one component of the procedure causes at most one error in each encoded block of cubits output from the procedure. The *threshold theorem of quantum computation* states that a quantum circuit with $p(n)$ gates may be simulated with an upper-bound probability of error ϵ using $O(\text{poly}(\log p(n)/\epsilon)p(n))$ gates on hardware with probability of physical error (from reasonable assumption) $p < p_{th}$ where p_{th} is some threshold probability that depends on the circuit. p satisfying $p < p_{th}$ yields the result that the logical error rate vanishes with increasing distance d [7].

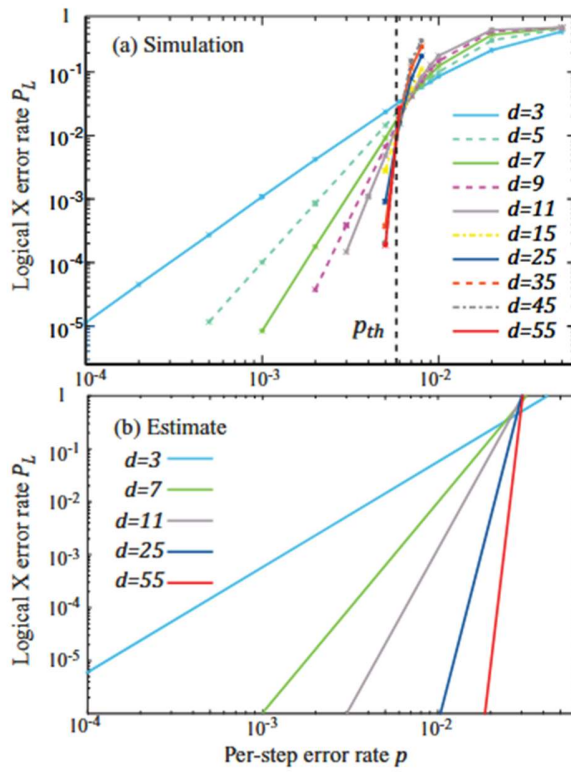


Figure 8 (a) example of a numerical simulation of surface code error rate with varying array distance d , with per step threshold 0.57%; (b) estimated error rates [7]. The threshold of a error correcting code depend on the noise model chosen, and in practice it can be found by plotting the Logical error rate against the physical error rate for different distances of the family of codes. The intersection between the lines gives an indication to the threshold.

2.4 Noise Models

Two noise models are employed in this study. The first is a simplified model designed to isolate the effect of hook errors: a single Pauli X error is applied to the ancilla qubit after each CNOT gate. By omitting all other error mechanisms, this model ensures that any difference in logical error rate between gate orders arises solely from differences in hook error propagation, allowing a clean comparison of their relative performance.

The second is a more realistic circuit-level noise model. Single-qubit depolarising noise with probability p_1 is applied after every single-qubit gate (e.g. Hadamard gates). Each measurement-with-reset (MR) operation is preceded and followed by an X error with probability p_1 , and the final measurement is preceded by an X error with the same probability. Two-qubit depolarising noise with probability p_2 is applied after every CNOT gate. The two-qubit depolarising channel with probability p_2 on qubits A and B is defined as

$$\mathcal{E}(\rho_{AB}) = (1 - p_2)\rho_{AB} + \frac{p_2}{15} \sum_{P \in \{I, X, Y, Z\}^{\otimes 2} \setminus \{II\}} P \rho_{AB} P^\dagger$$

where ρ_{AB} denotes the two-qubit density matrix of qubits A and B . This means that there are probabilities of $\frac{p_2}{15}$ for a 2-qubit error $\{I, X, Y, Z\}^{\otimes 2}$ to be applied after a CNOT gate (excluding II as it corresponds to no errors in either of the qubits).

Idle qubits — those not involved in any operation during a given timestep — are subject to single-qubit depolarising noise with a separately configurable idle error rate p_{idle} . For simplicity, the probabilities of error after single-qubit and two-qubit operations are set to be equal, i.e. $p = p_1 = p_2$. The model further allows the idle noise to be applied either during all timesteps, or only during timesteps that do not involve any CNOT gates, to assess the sensitivity of results to the assumed idle error behaviour. For the simulations of the 2D triangular colour code, the same circuit-level noise model is used, with the restriction that idle depolarising noise is applied during all timesteps without exception.

3 Methods

The aim of this work is to:

- 1. investigate circuit pattern optimisation strategies for the unrotated surface code and the triangular colour code that reduce logical error rates by reducing impact of hook error.
- 2. analyse the impact on logical error suppression and threshold behaviour and investigate under what situations is the trade off from the optimisation strategy beneficial, to inform scalable fault-tolerant quantum computing designs.
- 3. motivate detailed theoretical study of the interaction mechanisms of hook errors with other error channels by investigating how the presence of other error mechanisms, in particular idle errors, affect the performance ranking of the gate orders in the unrotated surface code and the 2D triangular colour code.

To achieve these aims, numerical simulations of the unrotated surface code and the triangular colour code are performed under multiple noise models. Threshold simulations under a circuit-level noise model are first carried out to establish the physical error rate regime for subsequent simulations(aim 2). Gate order schedules, which differ in the ordering of CNOT gates within each stabilizer measurement cycle and therefore in how hook errors propagate, are then simulated first under a simple noise model that isolates hook error effects(aims 1,3), and subsequently under a realistic circuit-level noise model with varying idle error rates to assess how the interaction between hook errors and other error mechanisms affects relative performance(aims 2,3).

3.1 Simulation Procedures

Numerical simulations of the unrotated surface code and the 2D triangular colour code ($d = 3, 5, 7$) under circuit level noise models are performed to obtain estimates of the code threshold. This ensures that the physical error rates p used in subsequent simulations are situated below the threshold.

Simulations are performed on variants of the $d = 7$ unrotated surface code, testing all possible CNOT gate orderings within each stabilizer, first under a simple noise model where only hook error propagation affects the relative performance among the gate orders, to identify the performance ranking. This ranking indicates which gate order best reduces the effect of hook errors in the absence of other error mechanisms.

Further simulations are performed under a more realistic circuit-level noise model, with varying idle error rates (probability of error for the qubits not involved in an operation in a timestep), to evaluate whether the relative performance of the gate orders changes when other error mechanisms are present .

Simulations on the $d = 5$ triangular colour code are also performed, under a realistic circuit-level noise model with low idle rate , considering only the Z-type stabilizers. The boundary stabilizer (weight-4) gate orders are fixed, and each symmetry-inequivalent class of bulk stabilizer (weight-6) gate orderings is simulated to identify the best-performing bulk gate order at a chosen value of physical error rate p below threshold.

The boundary gate orders are then enumerated to determine the overall best schedule that minimises the logical error rate. The schedules are grouped into statistically distinguishable performance clusters, given the number of shots used. Finally, the idle error rate is increased incrementally to evaluate its effect on the relative performance among the statistically distinguishable schedules.

The simulations are performed using Stim, an efficient stabilizer circuit simulator [9], with the python package Sinter used to manage sampling and collect statistics. The functions used for generating the circuits can be found in [25]. The number of shots simulated for the unrotated surface code threshold estimation under each of noise model is 10^6 , while for the subsequent simulations 5×10^6 shots are simulated for each case. For the 2D triangular colour code 5×10^6 shots are simulated for the threshold estimations, with 10^7 shots for subsequent simulations. The maximum error count is set at a value such that the shots simulated reaches the maximum value set. The decoder choice for the unrotated surface code is PyMatching [22], for the 2D triangular colour code is VibelSD [23].

3.2 Gate Order in Stabilizer Measurements

3.2.1 Stabilizer Gate Orders in the Unrotated Surface Code

Denote the CNOT gate orientations for the X-type stabilizers as follows: where the target qubit facing East as “1”, facing South as “2”, facing North as “3” and facing West as “4”. Similarly, for the Z type stabilizers denote where the control qubit facing East as “1”, facing South as “2”, facing North as “3” and facing West as “4”. Then each of the possible order of CNOT application for the X and Z type stabilizers are represented as permutations of $[1,2,3,4]$. The different type of stabilizers propagates different types of hook errors, which affects either of the logical X or logical Z operators. Depending on the choice of logical operator only one type of stabilizer gate order will affect the performance. In the following, the logical Z operator is chosen, thus the relevant gate orders for the simulations are that of the X type stabilizers. Consequently, the gate orders of the Z type stabilizers are kept as $[1,2,3,4]$.

Some permutations represent gate orders identical under symmetry of the qubit grid. For example, reflecting the grid about the y axis swaps the CNOT orientations 1 and 4, while keeping 2 and 3 unchanged ; reflecting the grid about the x axis swaps 2 and 3 and left 1 and 4 unchanged. The symmetry group of the unrotated surface code is $(\mathbb{Z}_2)^3$ generated by reflection about the x-axis R_x , reflection about the y-axis R_y , and time reversal symmetry T . The symmetry of the circuit reduces the number of equivalence classes of gate orders for a stabilizer type from $4! = 24$ to 4, namely the schedules $[1,2,3,4]$, $[1,2,4,3]$, $[1,4,2,3]$, $[2,1,4,3]$.

Table 1 The orientations of the CNOTs in the X-type stabilizers of the unrotated surface code and the corresponding key used.

Key	Δx	Δy	Direction
1	1	0	East
2	0	-1	South
3	0	1	North
4	-1	0	West

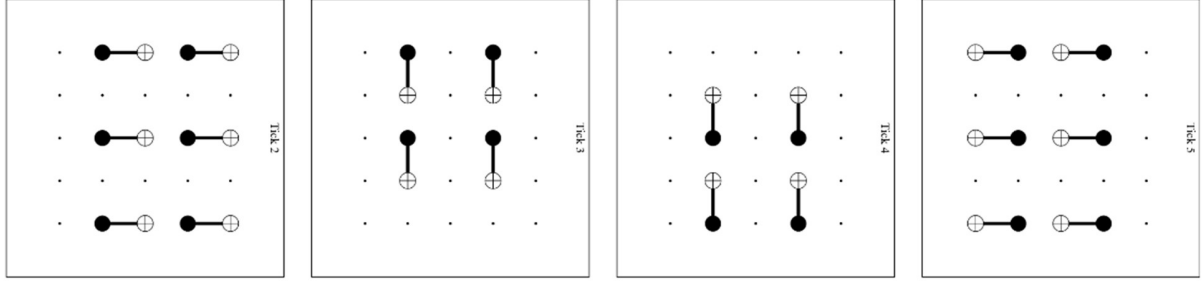


Figure 9 The orientation of CNOTs in the X-type stabilizers of the $d=3$ unrotated surface code, visualized over the grid of qubits. Each “tick” is a different timestep. From left to right are the CNOT orientations denoted 1,2,3,4.

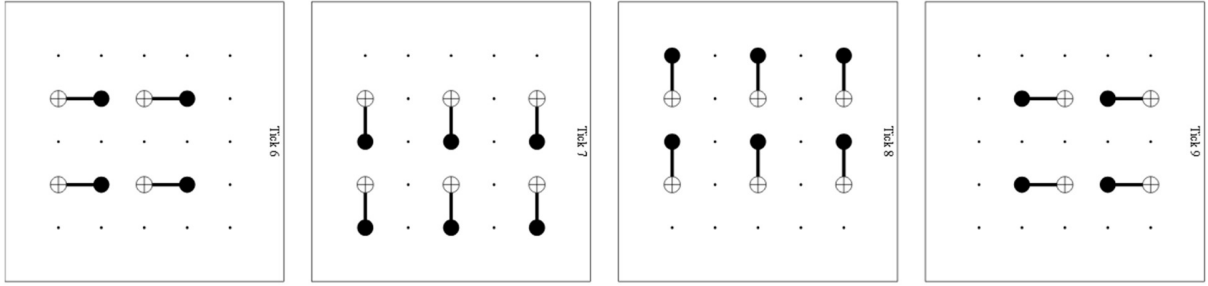


Figure 10 The orientation of CNOTs in the Z-type stabilizers of the $d=3$ unrotated surface code, visualized over the grid of qubits. From left to right are the orientations denoted 1,2,3,4.

3.2.2 Stabilizer Gate Orders in the 2D Triangular Colour Code

The error correction procedures X and Z errors are analogous [21], hence only the Z-type stabilizers are considered in the simulations for simplicity.

There are 6 possible CNOT orientations for the bulk stabilizers (weight-6). For the boundary stabilizers (weight-4) only 4 out of the 6 orientations are present, depending on which boundary it is situated at. We denote each possible orientations with numbers 1,2,3,4,5,6 such that the possible schedules for the stabilizers are represented as permutations of $(1,2,3,4,5,6)$, where it is set separately for the bulk and boundary stabilizers. The $d = 3$ triangular colour code is a special case where only weight-4 stabilizers are present. The relevant symmetry group is $\mathbb{Z}_3 \times \mathbb{Z}_2$, generated by R ($\frac{2\pi}{3}$ clockwise rotation), R^2 ($\frac{4\pi}{3}$ clockwise rotation) and time reversal symmetry T , which reduces the number of equivalence class for stabilizers of each weight from $6! = 720$ to 120. As an example, the table of gate order schedules equivalent to $[1,2,3,4,5,6]$ for both the bulk and boundary stabilizers can be seen below.

Table 2 The gate order schedules equivalent to $[1,2,3,4,5,6]$ for both the bulk and boundary stabilizers.

Bulk order	Boundary order	Operations applied
$[1,2,3,4,5,6]$	$[1,2,3,4,5,6]$	I
$[1,6,3,2,4,5]$	$[1,6,3,2,4,5]$	RT
$[5,1,2,4,3,6]$	$[5,1,2,4,3,6]$	R^2T
$[5,4,2,3,6,1]$	$[5,4,2,3,6,1]$	R
$[6,3,4,2,1,5]$	$[6,3,4,2,1,5]$	R^2
$[6,5,4,3,2,1]$	$[6,5,4,3,2,1]$	T

Table 3 The orientations of the CNOTs in the Z-type stabilizers of the 2D triangular colour code and the corresponding key used.

Key	Δx	Δy	Direction
1	1	0	East
2	0.5	1	SE
3	0.5	-1	NE
4	-1	0	West
5	-0.5	1	SW
6	-0.5	-1	NW

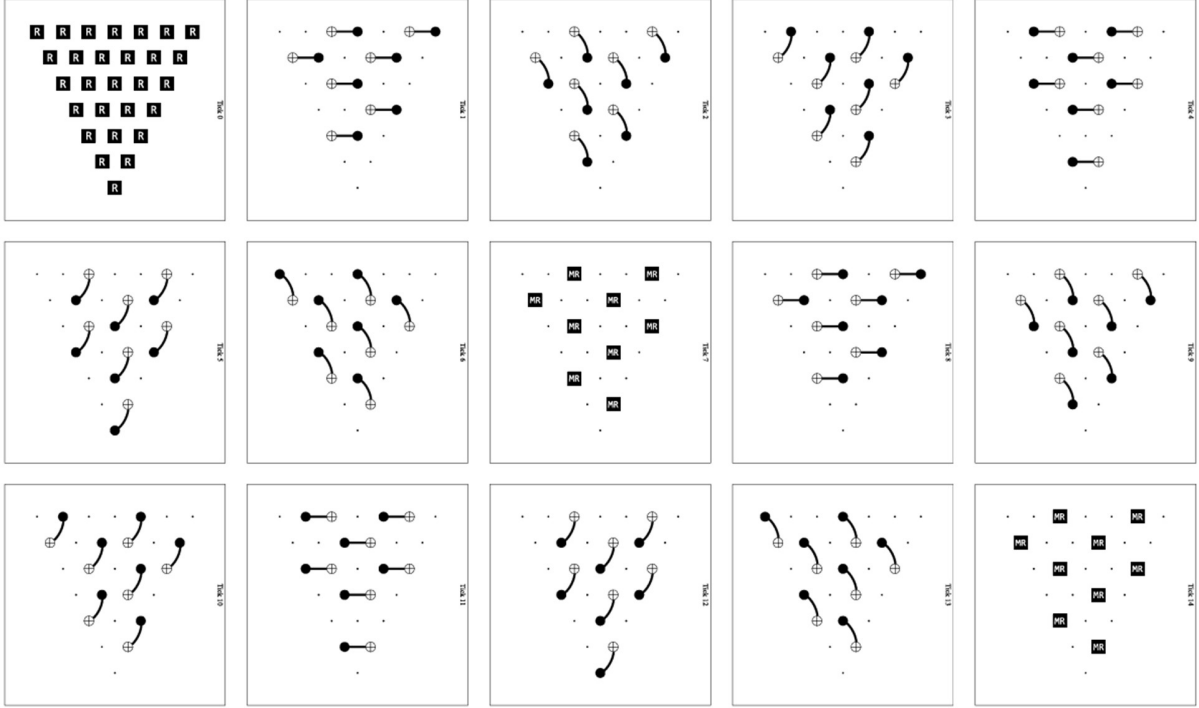


Figure 11 Implementation of the first two rounds of a $d=5$ triangular colour code with gate order $[1,2,3,4,5,6]$ on both the bulk and boundary stabilizers. The CNOT gates of a stabilizer is measured over 6 ticks(timesteps). Following the stabilizer measurement is a measurement reset(MR) on the ancilla qubits.

3.3 Statistical Distinguishability (2D Triangular Colour Code)

For the 2D triangular colour code, the gate orders simulated are grouped into statistically distinguishable clusters based on their logical error rates. For each simulated order, the logical error rate is estimated as $p_L \approx \hat{p} = n_{\text{errors}}/n_{\text{shots}}$, with a standard error of $\sigma = \sqrt{\hat{p}(1 - \hat{p})/n_{\text{shots}}}$. Orders are sorted by ascending $\hat{p}$ and grouped sequentially: an order is assigned to the current cluster if its logical error rate differs from that of the cluster's

lowest-performing anchor by less than $k\sigma_{\text{combined}}$, where $\sigma_{\text{combined}} = \sqrt{\sigma_{\text{anchor}}^2 + \sigma_{\text{order}}^2}$ is the combined standard error of the two estimates and $k = 2$ corresponds to a 95% confidence level. When an order exceeds this threshold, a new cluster is started with that order as the new anchor. Orders within the same cluster are therefore statistically indistinguishable from the cluster's worst performer at the 95% confidence level, while orders in different clusters are distinguishable.

4 Results

4.1 Surface Codes Results

4.1.1 d=3 Unrotated Surface Code Under Simple Noise Model

Logical error rates p_L of the d=3 unrotated surface code for each of the equivalence classes of gate order under the simple noise model is shown in (fig.12). As expected, when hook error propagation is the only mechanism leading to logical error, the performance of each of the gate order class which propagates hook errors differently is distinct, and the best performing class can be easily identified. The equivalence class represented by [1,2,3,4] performs worst under the simple noise model, followed by [1,2,4,3], [1,4,2,3] with [2,1,4,3] performing best with lowest p_L . This provides a baseline for the effect of hook error propagation of the different gate orders on p_L in the absence of other error mechanisms.

Surface Code Error Rates for different gate orders (Logical Z; d=3 ; simple noise model)

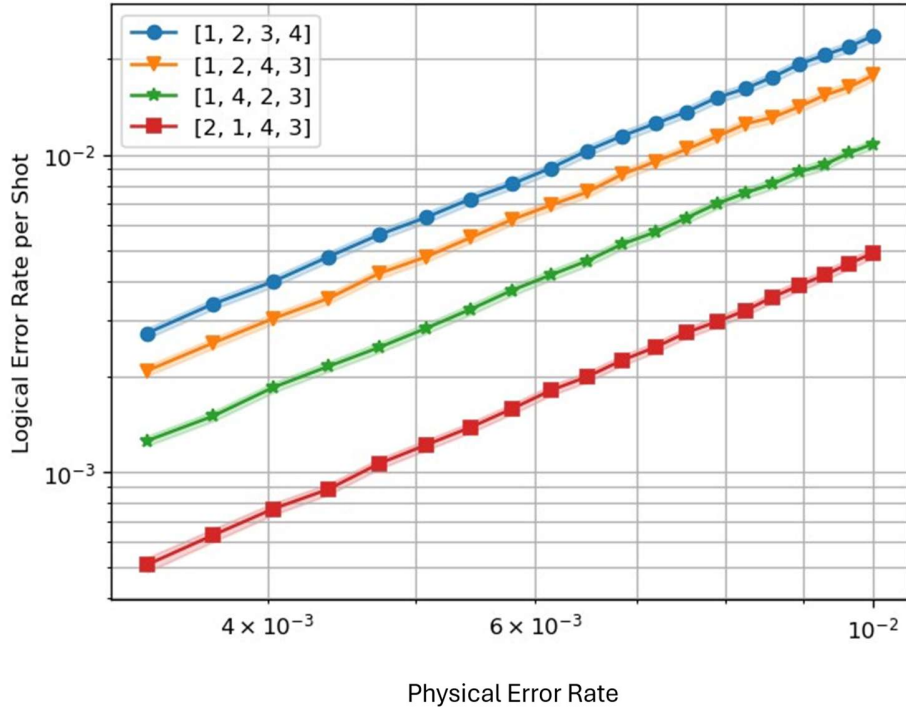


Figure 12 Logical error rates of the d=3 unrotated surface code for each of the equivalence classes of gate order, under the simple noise model where a Pauli X (bit flip) error of probability equal to the physical error rate p is placed on the ancilla qubit after each CNOT gates. The distinct level of performance can be seen clearly as expected, since the only mechanism leading to logical error for the simple noise model is the propagation of the hook errors.

The performance of all possible gate orders is plotted in (fig 13), showing 4 distinct performance levels and confirming the existence of 4 equivalence class under symmetry group $(\mathbb{Z}_2)^3$.

Surface Code Error Rates for different gate orders (Logical Z; d=3 ; simple noise model)

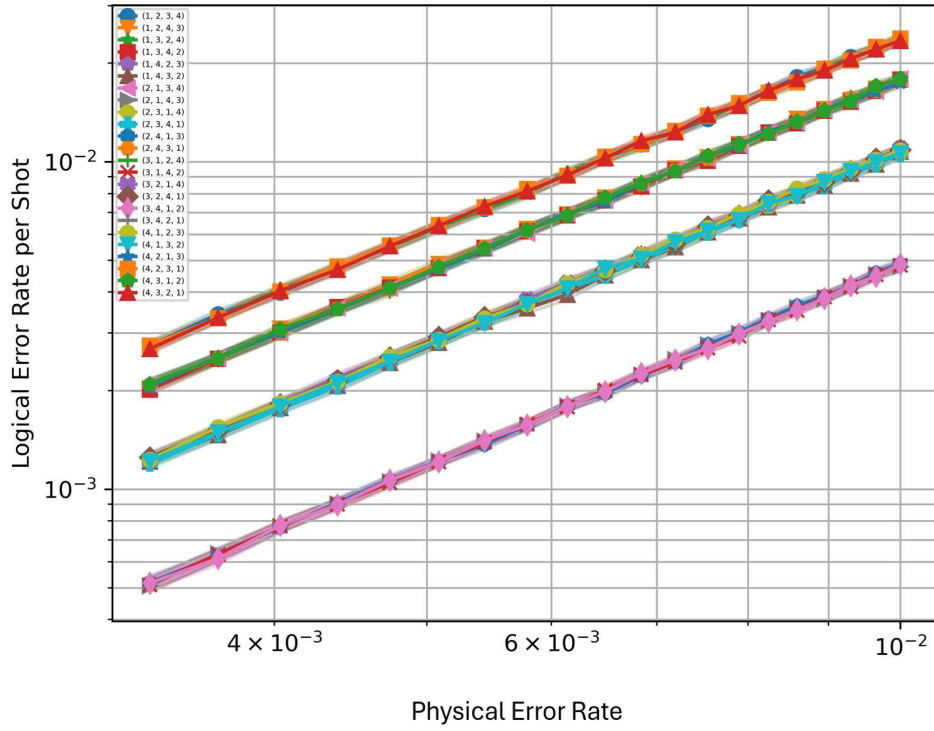
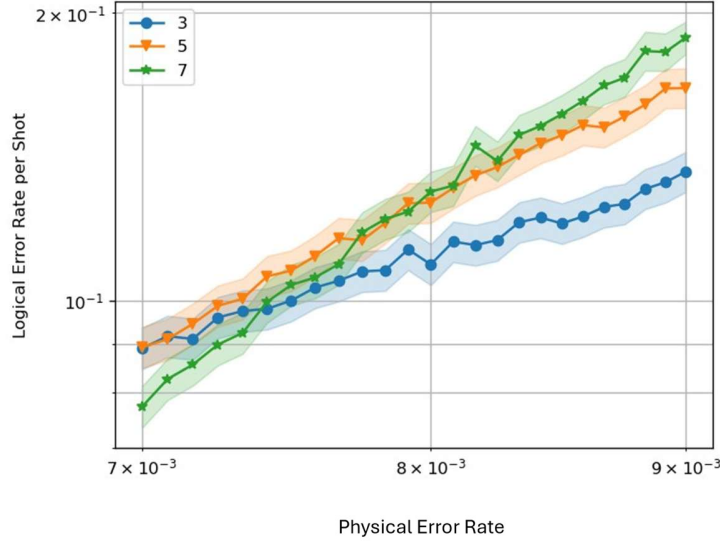


Figure 13 Explicit plot of all possible permutations of gate orders for the unrotated surface code, showing 4 distinct equivalent classes for hook error propagation under the simple noise model, agreeing with the expectation of 4 equivalence class under the symmetry group $(\mathbb{Z}_2)^3$

4.1.2 Unrotated Surface Code Under Realistic Noise Model

Code threshold under the realistic noise model is estimated from the intersection in the plot of p_L against p for different code distance, firstly with idle errors turned off during the ticks(timesteps) involving CNOT gates(fig.14). The idle error rate is chosen to be $p_{idle} = 0.1p$ to simulate a subdominant but non-negligible idle error process. This scale is plausible for superconducting qubit devices, where decoherence per gate-time is smaller than, but comparable to, two-qubit gate errors [7].

Surface Code Error Rates near threshold (Logical Z; X type gate order [1,2,3,4] ; $p_{idle}=0.1p$, no idle error in CX ticks)



Surface Code Error Rates near threshold (Logical Z; X type gate order [2,1,4,3] ; $p_{idle}=0.1p$, no idle error in CX ticks)

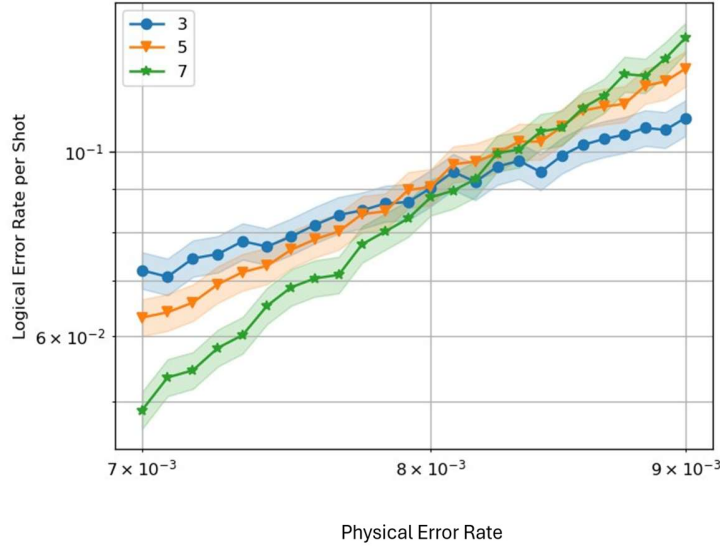


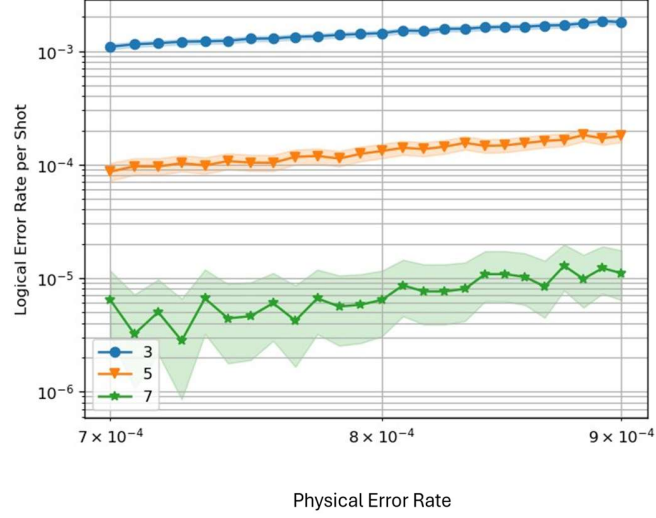
Figure 14 Threshold estimation for the unrotated surface code with X-type gate orders [1,2,3,4], [2,1,4,3]; $p_{idle} = 0.1p$ excluding timesteps involving CNOTs. The intersection of the lines for the different code distance gives an estimation of the threshold.

The thresholds for the different schedules are found to be different. Specifically for the equivalence class represented by gate order [1,2,3,4], which is the worst performing class with highest p_L , is found to be around 7.4×10^{-3} to 7.8×10^{-3} , whereas for the class represented by [2,1,4,3] the threshold is around 7.9×10^{-3} to 8.4×10^{-3} .

This change of threshold when changing gate order is a signature of changing effective code distance d_{eff} caused by the different propagation of faults through the syndrome extraction circuit, in particular the orientation and accumulation of hook errors induced by the CNOT ordering. More optimal gate orderings orient hook errors away from logical directions, preserving the code distance and leading to a higher observed threshold.

Both gate orders [1,2,3,4] and [2,1,4,3] shows higher logical error suppression with higher code distance for p below threshold (fig.15), as expected for below-threshold behaviour.

Surface Code Error Rates near 1/10 of threshold (Logical Z; X type gate order [1,2,3,4] ; $p_{\text{idle}}=0.1p$, no idle error in CX ticks)



Surface Code Error Rates near 1/10 of threshold (Logical Z; X type gate order [2,1,4,3] ; $p_{\text{idle}}=0.1p$, no idle error in CX ticks)

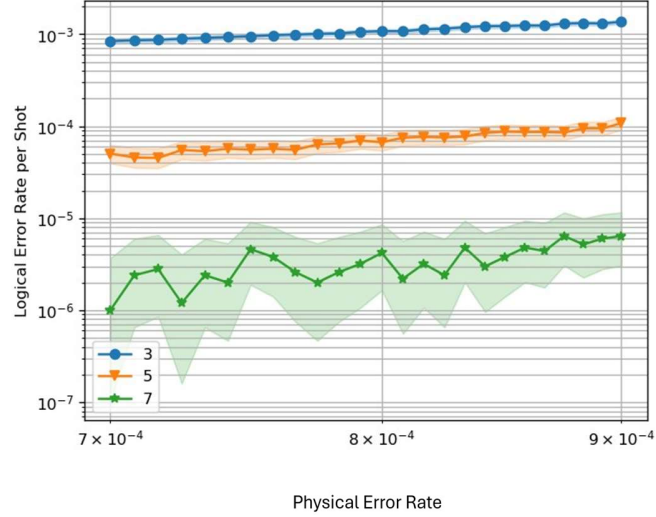
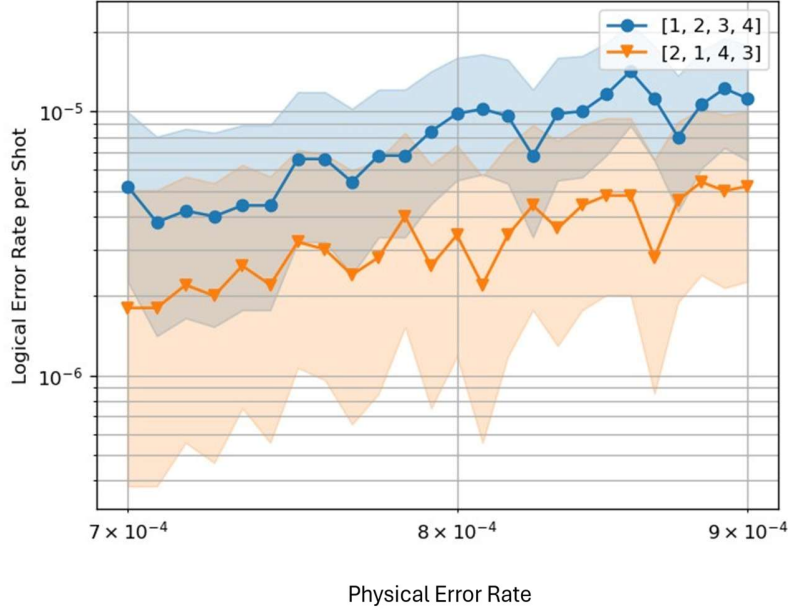


Figure 15 Below threshold logical error rates for the unrotated surface code with gate order [1,2,3,4],[2,1,4,3], under realistic noise model with idle error disabled during CNOT ticks; $d=3,5,7$. Higher code distance at below threshold result in higher error suppression. It is chosen to focus on around 1/10 of the threshold such that it gives a high error suppression factor.

p_L of the gate order [1,2,3,4] is found to be higher than that of the gate order [2,1,4,3], as shown in (fig.16), consistent with the baseline performance ranking obtained from the simulations under the simple noise model.

However, when comparing gate orders [1,4,2,3] and [2,1,4,3], it is found that under the realistic noise model the performance of the two gate orders is no longer distinguishable (fig.16), which indicate a non-trivial gate-order-dependent joint effect on p_L for hook errors and idle errors.

Surface Code Error Rates near 1/10 of threshold (Logical Z; $d=7$; $p_{\text{idle}}=0.1p$, no idle error in CX ticks)



Surface Code Error Rates near 1/10 of threshold (Logical Z; $d=7$; $p_{\text{idle}}=0.1p$, no idle error in CX ticks)

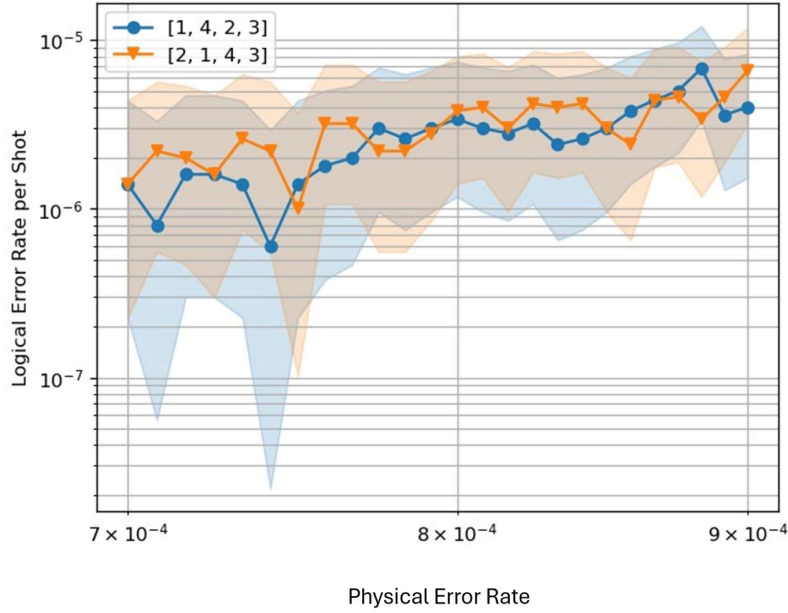
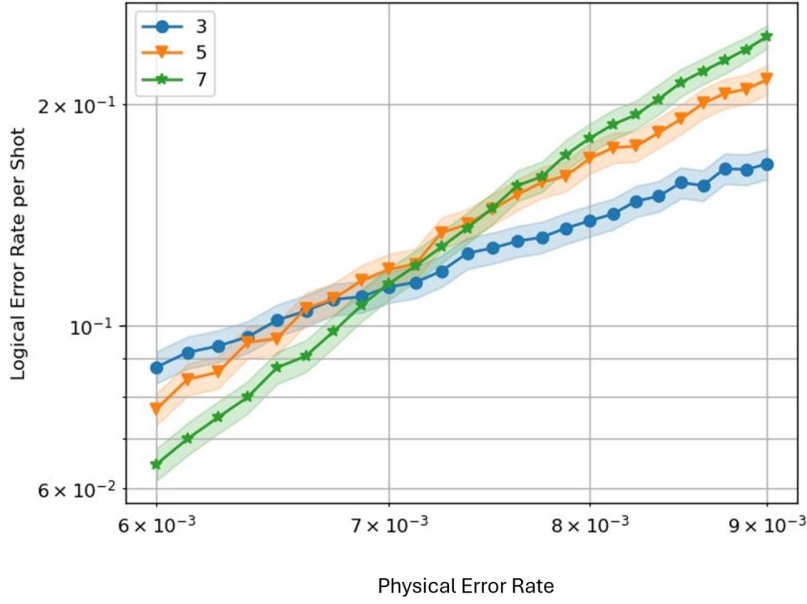


Figure 16 Comparison of logical error rates for the different gate orders of the $d=7$ unrotated surface code under the realistic noise model with idle error disabled during CNOT ticks. The gate order [1,2,3,4] which performs worst under the simple noise model remains consistently worse than the best performing gate order [2,1,4,3]. However, the gate order with intermediate performance under the simple noise model, [1,4,2,3], becomes indistinguishable to [2,1,4,3] under the realistic noise model with the chosen parameters, this demonstrates that **hook errors and idle errors have a non-trivial joint effect on logical error rate that depends on gate order.**

The idle error on CNOT ticks is then switched on such that at all time steps the idle rate $p_{idle} = 0.1p$. It is found that the threshold for each of the gate orders decreased (fig.17), which is unsurprising since introducing new errors will decrease the threshold.

Surface Code Error Rates near threshold (Logical Z; X type gate order [1,2,3,4] ; $p_{idle}=0.1p$ on all ticks)



Surface Code Error Rates near threshold (Logical Z; X type gate order [2,1,4,3] ; $p_{idle}=0.1p$ on all ticks)

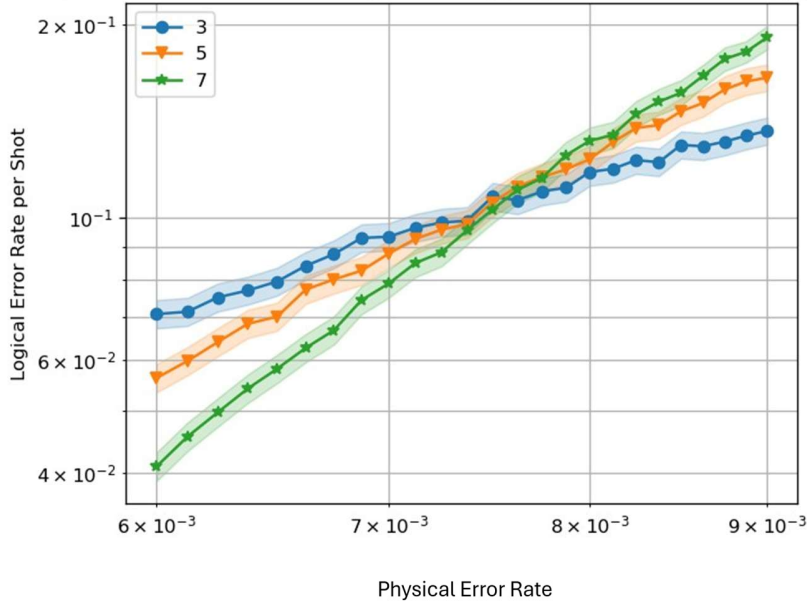


Figure 17 Threshold estimates for the unrotated surface code with X-type gate orders [1,2,3,4], [2,1,4,3]; $p_{idle} = 0.1p$ applied uniformly. The threshold decreases compared to when idle error is excluded on CNOT ticks.

The error rates below threshold for each of the gate orders is shown below (fig. 18). The gate orders (1,2,3,4) and (1,2,4,3) remains the worst and second worst performing, whereas the performance ranking of gate orders (1,4,2,3) and (2,1,4,3) appears swapped compared to the case in the simple noise model, with the gate order (1,4,2,3) now performing best with the lowest p_L .

Surface Code Error Rates near 1/10 of threshold (Logical Z; $d=7$; $p_{\text{idle}}=0.1p$ on all ticks)

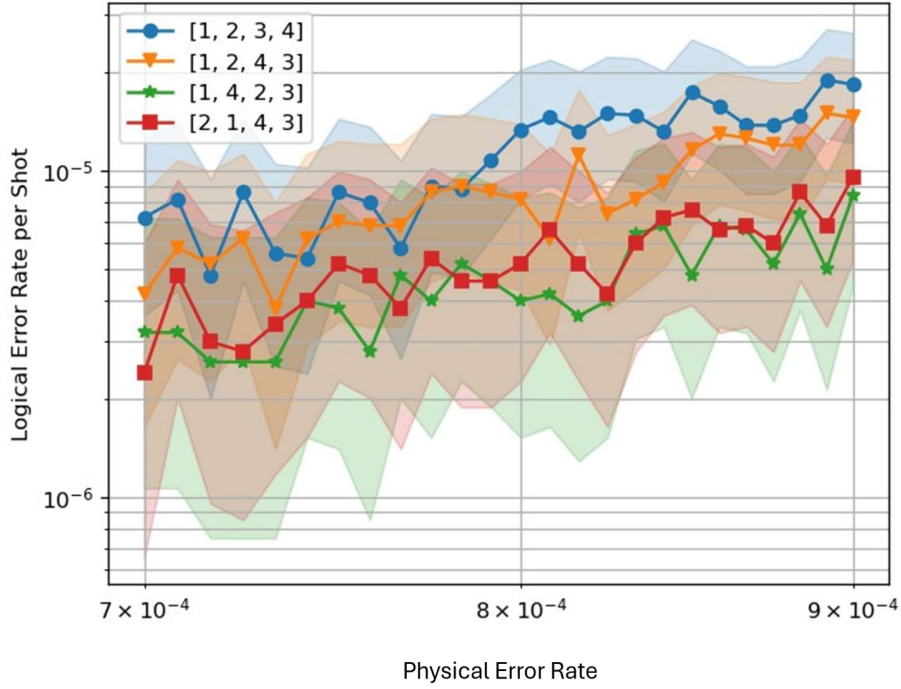


Figure 18 Comparison of logical error rates for the different gate orders of the $d=7$ unrotated surface code under the realistic noise model with idle error applied uniformly. The gate orders [1,2,3,4] and [1,2,4,3] remains the worst and second worst performing, whereas the performance ranking of gate orders [1,4,2,3] and [2,1,4,3] appear to have swapped.

4.2 Colour Codes Results

A below-threshold value of $p = 3 \times 10^{-4}$ is chosen, to investigate the performance ranking among the best three distinguishable overall schedules identified above as p_{idle} increases incrementally from $0.01p$ up to $0.2p$. This informed by threshold estimation of the code. The specific plots for $p_{idle} = 0.2p$ is shown in fig.19-21, where all thresholds estimate sits above 4×10^{-4} .

Table 4 below shows the statistically distinguishable performance clusters(as defined in section 3.3) for the different bulk gate orders with boundary order fixed at $[1,2,3,4,5,6]$, at a uniform idle noise level of $p_{idle} = 0.01p$ for every time step. The best performing bulk gate order is found to be $[1,3,4,2,6,5]$. Whereas table 5 shows the performance clusters for the different boundary gate orders with bulk order fixed at the best performing order $[1,3,4,2,6,5]$. The full dataset for all 120 bulk and boundary orders is attached in the appendix.

The best performing overall schedule at $p_{idle} = 0.01p$ is thus found to be {bulk, boundary}= $\{[1,3,4,2,6,5], [2,5,3,6,1,4]\}$, followed by $\{[1,3,4,2,6,5], [1,2,3,4,5,6]\}$, $\{[1,3,4,2,6,5], [1,5,3,2,6,4]\}$ and so on.

Table 4 The statistically distinguishable bulk order clusters with boundary order fixed at $[1,2,3,4,5,6]$. 8 clusters are identified with the best performing cluster having only one member, $[1,3,4,2,6,5]$.

Cluster	Size	Range of mean P_L	Representative bulk order
1	1	8.660×10^{-5}	$[1,3,4,2,6,5]$
2	57	$9.85 \times 10^{-5} - 1.069 \times 10^{-4}$	$[1,3,4,5,6,2]$
3	27	$1.077 - 1.168 \times 10^{-4}$	$[1,2,5,6,3,4]$
4	19	$1.173 - 1.266 \times 10^{-4}$	$[1,2,4,3,5,6]$
5	6	$1.290 - 1.384 \times 10^{-4}$	$[2,5,1,3,6,4]$
6	6	$1.423 - 1.531 \times 10^{-4}$	$[1,5,6,2,4,3]$
7	3	$1.538 - 1.576 \times 10^{-4}$	$[1,3,5,4,6,2]$
8	1	1.784×10^{-4}	$[1,2,5,3,6,4]$

Table 5 The statistically distinguishable boundary order clusters with bulk order fixed at [1, 3, 4, 2, 6, 5]. 6 clusters are identified with the best performing cluster represented by the boundary order [2, 5, 3, 6, 1, 4]. The overall best schedule for the triangular colour code can thus be identified as {[1, 3, 4, 2, 6, 5], [2, 5, 3, 6, 1, 4]}.

Cluster	Size	Range of mean P_L	Representative boundary order
1	2	$5.950 - 6.370 \times 10^{-5}$	[2,5,3,6,1,4]
2	30	$9.250 \times 10^{-5} - 1.010 \times 10^{-4}$	[1,2,3,4,5,6]
3	64	$1.015 - 1.102 \times 10^{-4}$	[1,5,3,2,6,4]
4	16	$1.110 - 1.206 \times 10^{-4}$	[1,2,6,4,5,3]
5	5	$1.227 - 1.302 \times 10^{-4}$	[1,5,6,4,2,3]
6	3	$1.445 - 1.530 \times 10^{-4}$	[1,5,2,3,6,4]

Threshold estimate for triangular colour code with idle rate = 0.2p

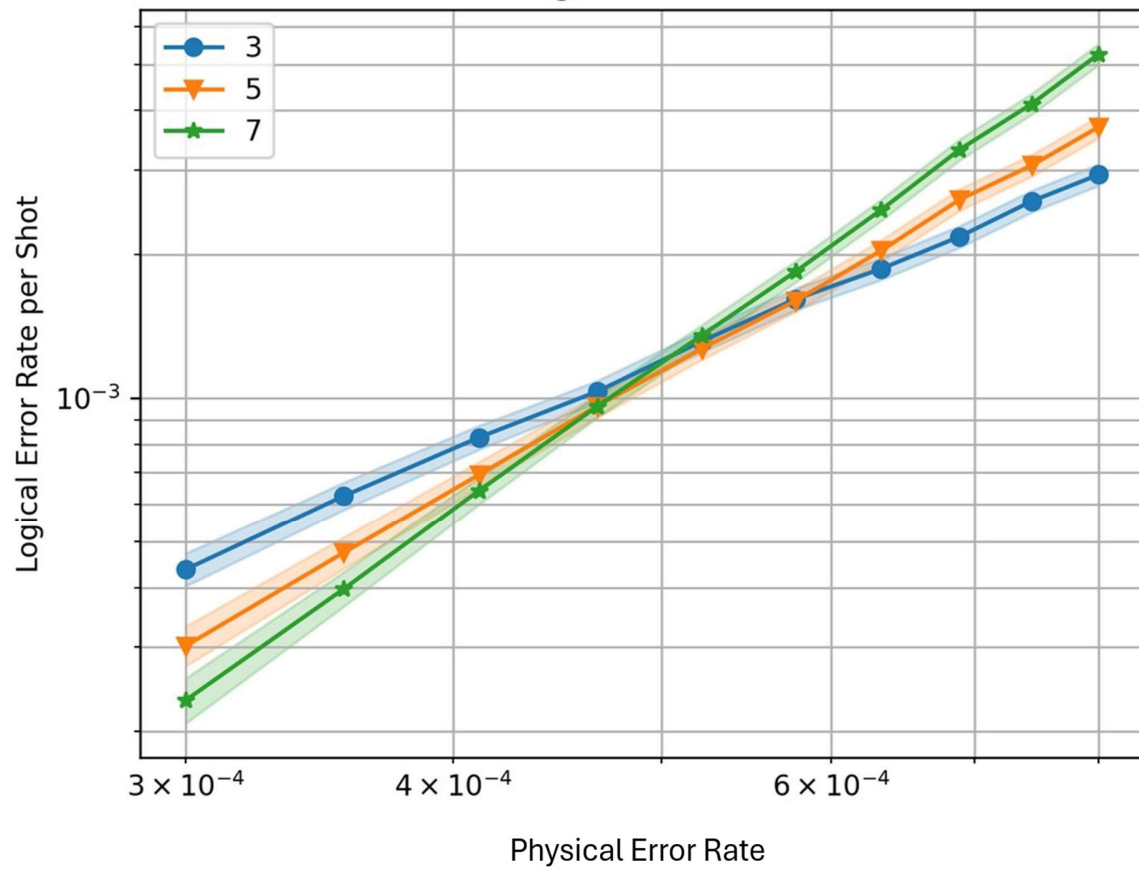


Figure 19 Threshold estimate for the triangular colour code with bulk gate order $[1,3,4,2,6,5]$, boundary gate order $[1,2,3,4,5,6]$.

Threshold estimate for triangular colour code with idle rate = 0.2p

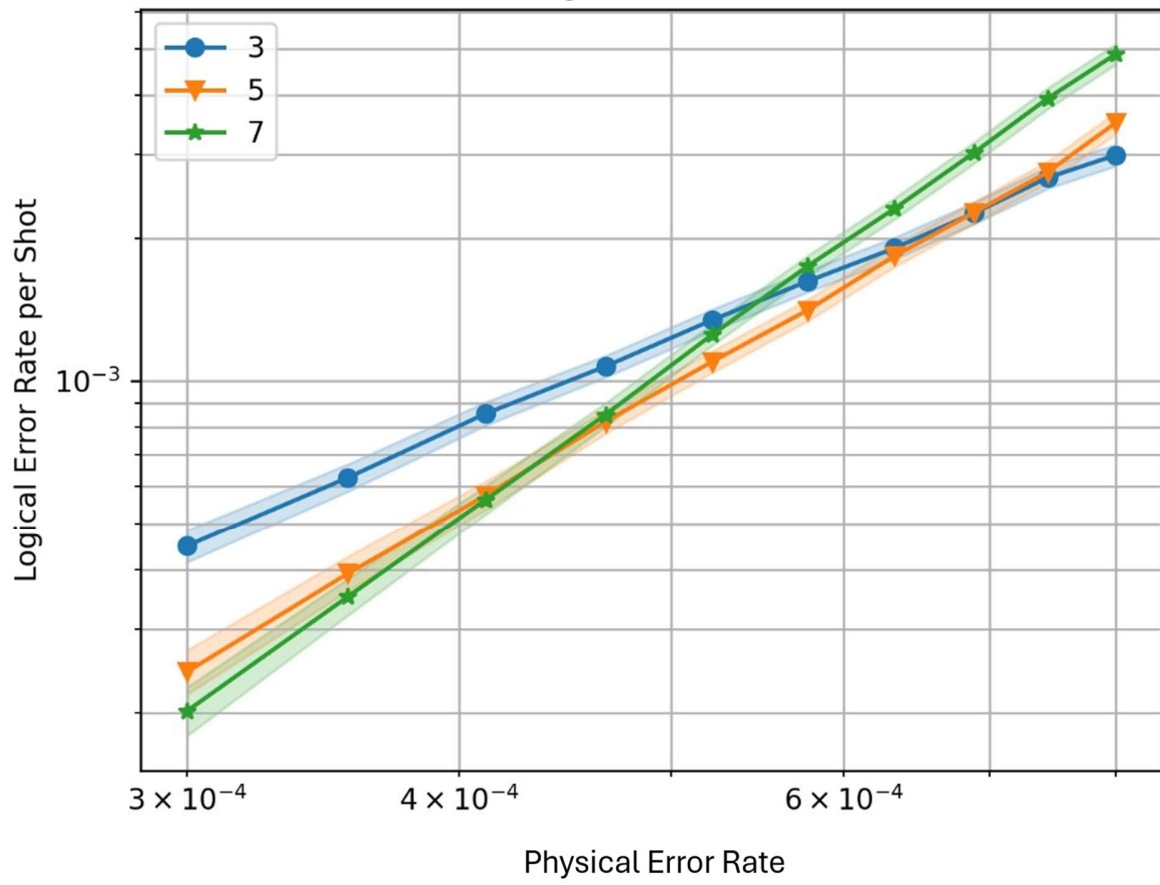


Figure 20 Threshold estimate for the triangular colour code with bulk gate order $[1,3,4,2,6,5]$, boundary gate order $[1,5,3,2,6,4]$.

Threshold estimate for triangular colour code with idle rate = 0.2p

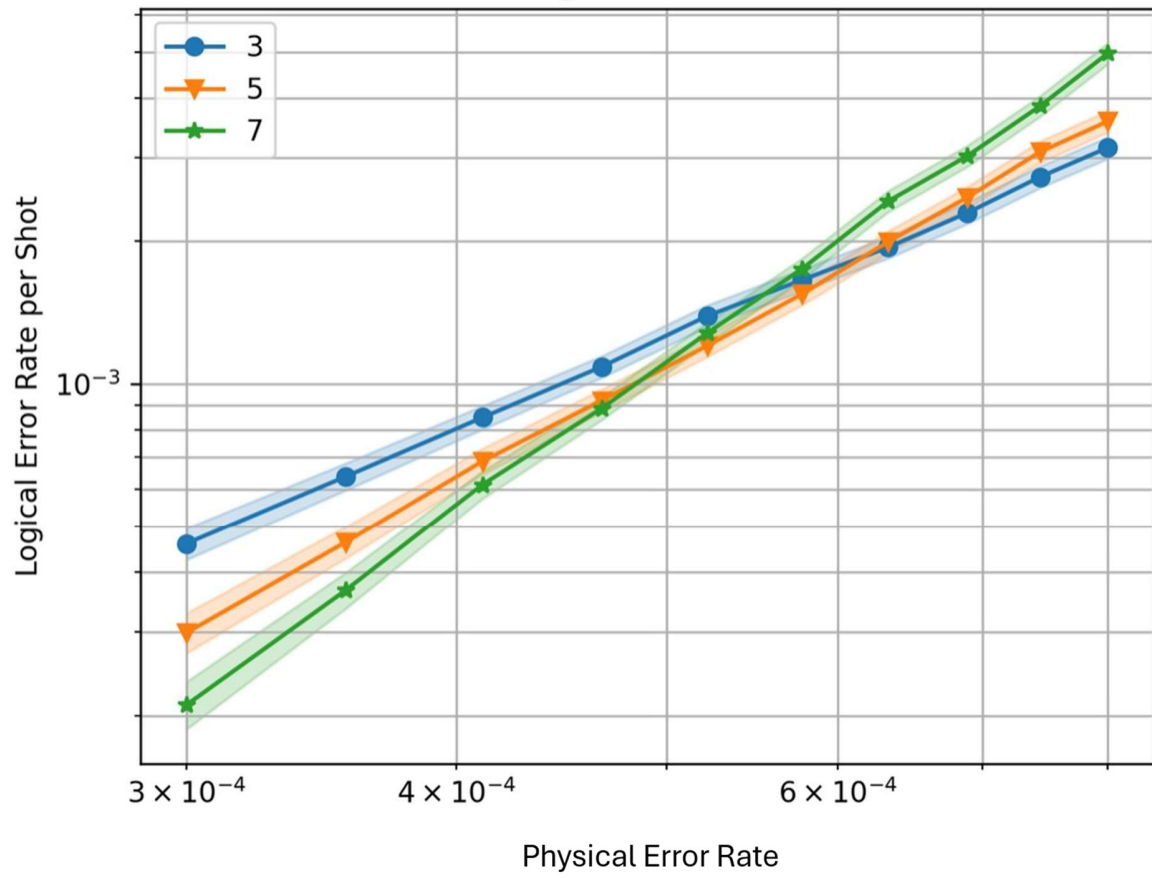


Figure 21 Threshold estimate for the triangular colour code with bulk gate order $[1,3,4,2,6,5]$, boundary gate order $[2,5,3,6,1,4]$.

Triangular colour code with bulk gate order(1,3,4,2,6,5) and different boundary gate order,physical error rate = 0.0003

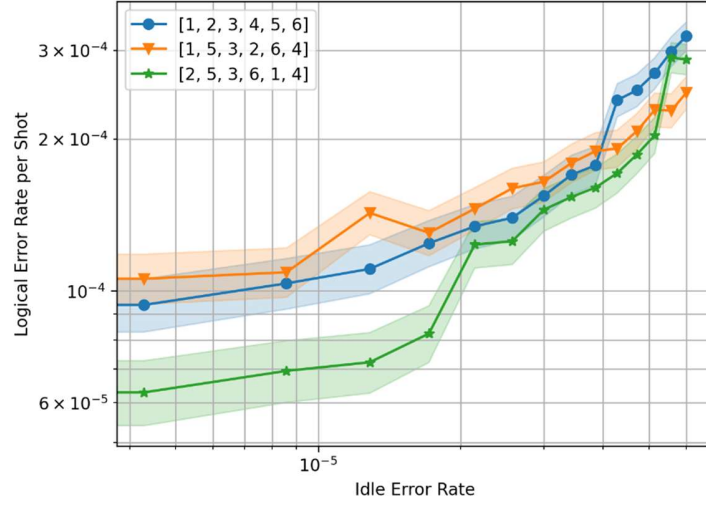


Figure 22 Logical error rates of the best 3 distinguishable schedules at $p_{idle} = 0.01p$ of the triangular colour code, with idle error rate increasing incrementally up to $0.2p$, at $p = 3 \times 10^{-4}$.

It can be seen from the intersection of the lines representing P_L at $p = 3 \times 10^{-4}$ as a function of p_{idle} (Fig.22) that the performance ranking among the schedules change depending on the level of p_{idle} relative to p , similar to the case in the unrotated surface code, again demonstrating non-trivial joint effect of gate orderings(which affect hook errors) and idle errors on P_L .

5 Discussion

The results of this study demonstrate that the relative performance of gate orders is not fixed but depends on the noise model under which the code operates. Under the simple noise model, where only hook error propagation contributes to differences in logical error rate, a clear performance ranking is established. However, this ranking does not remain stable as the noise model is made more realistic: the introduction of idle errors and the variation of the idle error rate both produce changes in the relative performance of gate orders in the unrotated surface code. This indicates that the interaction between hook errors and other noise channels is non-trivial, where the combined effect on logical error rate is not a simple additive contribution from each mechanism independently, but reflects more complex interaction dynamics between the error channels.

A particularly illustrative example of this interaction is the change in relative performance observed when idle errors on CX ticks are toggled on and off. Since the two gate orders in question perform identically in the absence of idle errors on CX ticks, any difference in logical error rate that emerges when such errors are introduced must arise specifically from differences in how the two schedules expose qubits to idle errors during CX-heavy timesteps. The gate orders differ in which qubits are idle at which points in the circuit, and those idle qubits occupy different positions in the error propagation graph depending on the schedule. The resulting performance change therefore suggests that the role of idle errors in contributing to logical failures is gate-order-dependent in a way that is coupled to the hook error structure of the circuit, rather than being a uniform additive penalty across all orders.

A practically significant finding for the unrotated surface code is that the gate order compatible with interleaving of X and Z stabilizer measurements, order (1,2,3,4), consistently ranks the worst in terms of hook error propagation. This has a concrete implication: for hardware platforms with sufficiently low idle error rates, a gate order that is incompatible with interleaving but better suited to hook error mitigation can still achieve a lower logical error rate overall, despite the longer circuit it requires. Ion trap quantum computers are a candidate platform in this regard, as they are characterised by significantly lower idle error rates per gate time than superconducting architectures, making the trade-off between circuit depth and hook error suppression more likely to be favourable. A detailed assessment of whether this trade-off is beneficial for a specific ion trap device would require a hardware-specific noise model and is left for future work.

The observation that gate order performance rankings change with varying idle error rates is not unique to the unrotated surface code. Qualitatively similar behaviour is observed in the 2D triangular colour code simulations. The fact that this phenomenon appears across two

structurally distinct codes — one based on weight-4 stabilizers on a square lattice and one involving weight-6 bulk stabilizers on a triangular geometry, suggests that the interaction between hook errors and idle errors could be a general feature of stabilizer circuits rather than an artefact of a particular code structure. This motivates broader investigation of the interaction mechanisms across different code families and noise regimes.

Several directions for future work follow naturally from these results. A theoretical analysis of the mechanisms by which hook errors and idle errors interact within stabilizer circuits would provide a principled basis for predicting performance rankings without exhaustive numerical simulation and could yield general design rules for gate order optimisation. For the unrotated surface code specifically, it would be valuable to determine the upper bound on the idle error rate below which the 8-tick schedule — incompatible with interleaving but optimal for hook error suppression — remains beneficial over the interleaving-compatible schedule, evaluated under a realistic hardware-specific noise model. Finally, the gate order scheduling study presented here considered a simplified noise model with probability of 1-qubit and 2-qubit error rates set to be equal, which is not the case generally. Repeating this analysis under a hardware-specific noise model would allow more direct assessment of the practical relevance of the identified optimal schedules.

6 Summary and Conclusions

This work investigated gate order optimisation as a strategy for reducing logical error rates through hook error suppression in the unrotated surface code and the 2D triangular colour code, using numerical simulation via Stim.

For the unrotated surface code, threshold simulations confirmed physical error rates used are below-threshold. Systematic simulation of all CNOT gate orderings established that the interleaving-compatible order [1,2,3,4] consistently performs worst in terms of hook error suppression for the values of p_{idle} simulated, implying that a non-interleaving schedule optimised for hook error mitigation can achieve lower logical error rates on platforms with sufficiently low idle error rates, with ion trap quantum computers identified as a candidate platform. The performance ranking was found to change with varying idle error rates, demonstrating that hook errors and idle errors interact non-trivially in a gate-order-dependent manner — a conclusion further supported by ranking changes observed when idle errors during CNOT timesteps were toggled.

For the 2D triangular colour code, the optimal gate order schedule was identified and all schedules grouped into statistically distinguishable performance clusters. Ranking changes with varying idle error rate were again observed, consistent with the surface code findings and suggesting the hook error-idle error interaction could be a general feature of stabilizer circuits.

These results confirm that gate order optimisation is a practical strategy for improving logical error suppression without increasing qubit overhead, while the observed noise-model dependence of optimal schedules motivates theoretical investigation of hook error interaction mechanisms and hardware-specific studies to determine the conditions under which optimised schedules provide a practical advantage.

References

- [1] Gottesman, D., 2009. An introduction to quantum error correction and fault-tolerant quantum computation. *arXiv preprint arXiv:0904.2557*.
- [2] Boykin, P.O., Mor, T., Pulver, M., Roychowdhury, V. and Vatan, F., 1999. On universal and fault-tolerant quantum computing. *arXiv preprint quant-ph/9906054*.
- [3] Calderbank, A.R. and Shor, P.W., 1996. Good quantum error-correcting codes exist. *Physical Review A*, 54(2), p.1098.
- [4] Chao, R. and Reichardt, B.W., 2020. Flag fault-tolerant error correction for any stabilizer code. *PRX Quantum*, 1(1), p.010302.
- [5] Nielsen, M.A. and Chuang, I.L., 2010. *Quantum computation and quantum information*. Cambridge university press.
- [6] Wootters, W.K. and Zurek, W.H., 1982. A single quantum cannot be cloned. *Nature*, 299(5886), pp.802-803.
- [7] Fowler, A.G., Mariantoni, M., Martinis, J.M. and Cleland, A.N., 2012. Surface codes: Towards practical large-scale quantum computation. *Physical Review A—Atomic, Molecular, and Optical Physics*, 86(3), p.032324.
- [8] Dennis, E., Kitaev, A., Landahl, A. and Preskill, J., 2002. Topological quantum memory. *Journal of Mathematical Physics*, 43(9), pp.4452-4505.
- [9] Gidney, C., 2021. Stim: a fast stabilizer circuit simulator. *Quantum*, 5, p.497.
- [10] Lomont, C., 2003. Quantum circuit identities. *arXiv preprint quant-ph/0307111*.
- [11] Yoder, T.J. and Kim, I.H., 2017. The surface code with a twist. *Quantum*, 1, p.2.
- [12] Grans-Samuelsson, L., Mishmash, R.V., Aasen, D., Knapp, C., Bauer, B., Lackey, B., da Silva, M.P. and Bonderson, P., 2024. Improved pairwise measurement-based surface code. *Quantum*, 8, p.1429.
- [13] Google Quantum AI and Collaborators. Quantum error correction below the surface code threshold. *Nature* 638, 920–926 (2025). <https://doi.org/10.1038/s41586-024-08449-y>
- [14] Lloyd, S. Universal quantum simulators. *Science* 273, 1073–1078 (1996)
- [15] Viszlai, J., Maurya, S., Tannu, S., Martonosi, M. and Chong, F.T., 2026. PropHunt: Automated Optimization of Quantum Syndrome Measurement Circuits. *arXiv preprint arXiv:2601.17580*.

- [16] Kubica, A. and Beverland, M.E., 2015. Universal transversal gates with color codes: A simplified approach. *Physical Review A*, 91(3), p.032330.
- [17] Bombín, H., 2013. An introduction to topological quantum codes. *arXiv preprint arXiv:1311.0277*.
- [18] Kubica, A. and Beverland, M.E., 2015. Universal transversal gates with color codes: A simplified approach. *Physical Review A*, 91(3), p.032330.
- [19] Lacroix, N., Bourassa, A., Heras, F.J., Zhang, L.M., Bausch, J., Senior, A.W., Edlich, T., Shutty, N., Sivak, V., Bengtsson, A. and McEwen, M., 2025. Scaling and logic in the color code on a superconducting quantum processor. *Nature*, pp.1-3.
- [20] Baireuther, P., Caio, M.D., Criger, B., Beenakker, C.W. and O'Brien, T.E., 2019. Neural network decoder for topological color codes with circuit level noise. *New Journal of Physics*, 21(1), p.013003.
- [21] Chamberland, C., Kubica, A., Yoder, T.J. and Zhu, G., 2020. Triangular color codes on trivalent graphs with flag qubits. *New Journal of Physics*, 22(2), p.023019.
- [22] Higgott, O., 2022. Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching. *ACM Transactions on Quantum Computing*, 3(3), pp.1-16.
- [23] Koutsoumpas, S., Noszko, T., Sayginel, H., Webster, M. and Roffe, J., 2025. Colour codes reach surface code performance using vibe decoding. *arXiv preprint arXiv:2508.15743*.
- [24] O'Rourke, A.R. and Devitt, S., 2025. Compare the pair: Rotated versus unrotated surface codes at equal logical error rates. *Physical Review Research*, 7(3), p.033074.
- [25] <https://github.com/carson6260/PHAS0097-project-repository-hook-errors-in-QEC->

Appendix

ABC Theorem

It can be shown [5] that any unitary 2x2 matrices U and hence single qubit quantum gates can be decomposed as $U = e^{i\alpha}AXBXC = e^{i\alpha}R_n(\beta)R_m(\gamma)R_n(\delta)$, where X is the Pauli X matrix and A, B, C are some single qubit operations represented by 2x2 matrices such that $ABC = I$, and $R_j(\theta)$ are rotation operators defined as:

$$R_{\hat{n}}(\delta) = e^{-i\frac{\delta}{2}\sigma \cdot \hat{n}}$$

Where $\sigma = e_x X + e_y Y + e_z Z$.

Using the expression of U above, one can deduce that if the Pauli X gates are applied only when a control qubit is set, that is, a CNOT, then it produces the effect of a controlled- U operation: where $U = e^{i\alpha}AXBXC$ is applied to the target if target is set, and no action otherwise.

Universal Quantum Computation

A universal set is a set of quantum gates that act as a basis such that other quantum gates can be constructed as a combination of these gates. For example, the Hadamard gate, $\frac{\pi}{8}$ gate and the CNOT gate $\{H, T, C^1(X)\}$ forms a universal set that can be implemented in a fault tolerant way[2].

Full Results for The Triangular Colour Code

Table 6 The full result of the triangular colour code with fixed boundary order (1,2,3,4,5,6) at $p_{idle} = 0.01p$ and $p = 3 \times 10^{-4}$.

Cluster	Bulk Gate Order	pL	sigma	95% CI Lower	95% CI Upper
1	[1,3,4,2,6,5]	0.0000866	0.000002943	0.00008071	0.00009249
2	[1,3,4,5,6,2]	0.0000985	0.000003138	0.00009222	0.0001048
2	[1,2,6,5,3,4]	0.0000986	0.00000314	0.00009232	0.0001049
2	[1,6,2,3,5,4]	0.0000987	0.000003142	0.00009242	0.000105
2	[2,1,4,5,6,3]	0.0000997	0.000003157	0.00009339	0.000106
2	[1,3,2,5,6,4]	0.0000998	0.000003159	0.00009348	0.0001061
2	[2,1,6,5,3,4]	0.0001001	0.000003164	0.00009377	0.0001064
2	[1,5,6,4,2,3]	0.0001003	0.000003167	0.00009397	0.0001066
2	[1,2,5,4,6,3]	0.0001007	0.000003173	0.00009435	0.000107
2	[2,1,3,6,5,4]	0.0001008	0.000003175	0.00009445	0.0001071

2	[1,2,6,5,4,3]	0.0001009	0.000003176	0.00009455	0.0001073
2	[1,3,2,5,4,6]	0.0001009	0.000003176	0.00009455	0.0001073
2	[1,3,6,4,5,2]	0.000101	0.000003178	0.00009464	0.0001074
2	[1,6,2,5,4,3]	0.0001011	0.000003179	0.00009474	0.0001075
2	[1,6,2,5,3,4]	0.0001012	0.000003181	0.00009484	0.0001076
2	[2,4,5,6,1,3]	0.0001012	0.000003181	0.00009484	0.0001076
2	[1,6,5,4,3,2]	0.0001015	0.000003186	0.00009513	0.0001079
2	[1,5,6,4,3,2]	0.0001015	0.000003186	0.00009513	0.0001079
2	[2,6,4,1,5,3]	0.0001016	0.000003187	0.00009523	0.000108
2	[1,3,6,5,2,4]	0.0001017	0.000003189	0.00009532	0.0001081
2	[1,3,4,2,5,6]	0.0001017	0.000003189	0.00009532	0.0001081
2	[1,3,4,6,5,2]	0.0001019	0.000003192	0.00009552	0.0001083
2	[2,4,6,5,1,3]	0.0001019	0.000003192	0.00009552	0.0001083
2	[1,5,2,6,3,4]	0.0001019	0.000003192	0.00009552	0.0001083
2	[1,2,3,6,5,4]	0.0001019	0.000003192	0.00009552	0.0001083
2	[1,3,4,5,2,6]	0.0001025	0.000003201	0.0000961	0.0001089
2	[2,5,6,3,1,4]	0.0001026	0.000003203	0.00009619	0.000109
2	[1,3,2,6,5,4]	0.0001029	0.000003208	0.00009648	0.0001093
2	[2,3,5,6,1,4]	0.000103	0.000003209	0.00009658	0.0001094
2	[1,6,3,2,5,4]	0.0001032	0.000003212	0.00009678	0.0001096
2	[1,2,3,4,5,6]	0.0001035	0.000003217	0.00009707	0.0001099
2	[1,5,3,6,4,2]	0.0001036	0.000003219	0.00009716	0.00011
2	[1,6,3,5,2,4]	0.0001036	0.000003219	0.00009716	0.00011
2	[1,4,5,6,2,3]	0.0001036	0.000003219	0.00009716	0.00011
2	[1,4,2,6,3,5]	0.0001039	0.000003223	0.00009745	0.0001103
2	[1,2,3,5,4,6]	0.000104	0.000003225	0.00009755	0.0001104
2	[1,2,5,3,4,6]	0.0001042	0.000003228	0.00009774	0.0001107
2	[2,3,6,5,1,4]	0.0001045	0.000003232	0.00009804	0.000111
2	[1,4,6,5,3,2]	0.0001047	0.000003236	0.00009823	0.0001112
2	[1,6,2,4,5,3]	0.0001048	0.000003237	0.00009833	0.0001113
2	[2,1,5,6,3,4]	0.0001048	0.000003237	0.00009833	0.0001113
2	[1,2,6,3,5,4]	0.000105	0.00000324	0.00009852	0.0001115
2	[1,3,2,4,5,6]	0.0001051	0.000003242	0.00009862	0.0001116
2	[2,6,1,4,5,3]	0.0001053	0.000003245	0.00009881	0.0001118
2	[1,5,2,6,4,3]	0.0001053	0.000003245	0.00009881	0.0001118
2	[1,2,4,5,3,6]	0.0001055	0.000003248	0.000099	0.000112
2	[1,6,5,2,3,4]	0.0001057	0.000003251	0.0000992	0.0001122
2	[1,6,3,4,5,2]	0.0001058	0.000003253	0.00009929	0.0001123
2	[2,1,6,5,4,3]	0.0001059	0.000003254	0.00009939	0.0001124
2	[1,6,3,5,4,2]	0.000106	0.000003256	0.00009949	0.0001125
2	[2,5,3,6,1,4]	0.000106	0.000003256	0.00009949	0.0001125
2	[2,1,5,6,4,3]	0.0001061	0.000003257	0.00009959	0.0001126
2	[1,4,6,5,2,3]	0.0001062	0.000003259	0.00009968	0.0001127

2	[1,2,3,5,6,4]	0.0001064	0.000003262	0.00009988	0.0001129
2	[2,1,6,4,5,3]	0.0001065	0.000003263	0.00009997	0.000113
2	[1,4,3,6,5,2]	0.0001069	0.000003269	0.0001004	0.0001134
2	[2,4,6,1,5,3]	0.0001069	0.000003269	0.0001004	0.0001134
2	[1,4,3,2,5,6]	0.0001069	0.000003269	0.0001004	0.0001134
3	[1,2,5,6,3,4]	0.0001077	0.000003282	0.0001011	0.0001143
3	[2,4,1,6,5,3]	0.0001078	0.000003283	0.0001012	0.0001144
3	[1,4,3,5,2,6]	0.000108	0.000003286	0.0001014	0.0001146
3	[1,2,6,4,5,3]	0.000108	0.000003286	0.0001014	0.0001146
3	[2,5,3,1,6,4]	0.0001082	0.000003289	0.0001016	0.0001148
3	[1,4,2,5,6,3]	0.0001085	0.000003294	0.0001019	0.0001151
3	[1,5,4,6,2,3]	0.0001087	0.000003297	0.0001021	0.0001153
3	[1,6,4,5,3,2]	0.000109	0.000003301	0.0001024	0.0001156
3	[1,6,4,5,2,3]	0.0001091	0.000003303	0.0001025	0.0001157
3	[1,5,3,6,2,4]	0.0001092	0.000003304	0.0001026	0.0001158
3	[1,3,6,5,4,2]	0.0001093	0.000003306	0.0001027	0.0001159
3	[1,4,2,6,5,3]	0.0001094	0.000003307	0.0001028	0.000116
3	[1,6,5,4,2,3]	0.0001097	0.000003312	0.0001031	0.0001163
3	[1,3,5,2,6,4]	0.0001098	0.000003313	0.0001032	0.0001164
3	[1,6,4,3,5,2]	0.0001107	0.000003327	0.000104	0.0001174
3	[1,4,5,6,3,2]	0.0001118	0.000003343	0.0001051	0.0001185
3	[1,6,5,3,4,2]	0.0001118	0.000003343	0.0001051	0.0001185
3	[1,6,5,2,4,3]	0.0001126	0.000003355	0.0001059	0.0001193
3	[1,5,4,6,3,2]	0.0001127	0.000003357	0.000106	0.0001194
3	[1,3,4,6,2,5]	0.0001131	0.000003363	0.0001064	0.0001198
3	[1,3,5,6,2,4]	0.0001131	0.000003363	0.0001064	0.0001198
3	[1,4,3,2,6,5]	0.0001146	0.000003385	0.0001078	0.0001214
3	[2,1,3,5,6,4]	0.000116	0.000003406	0.0001092	0.0001228
3	[1,4,2,3,6,5]	0.0001166	0.000003414	0.0001098	0.0001234
3	[1,3,5,6,4,2]	0.0001167	0.000003416	0.0001099	0.0001235
3	[1,5,4,2,6,3]	0.0001168	0.000003417	0.00011	0.0001236
3	[1,4,5,3,6,2]	0.0001168	0.000003417	0.00011	0.0001236
4	[1,2,4,3,5,6]	0.0001173	0.000003425	0.0001105	0.0001241
4	[1,4,6,3,5,2]	0.0001176	0.000003429	0.0001107	0.0001245
4	[1,3,5,4,2,6]	0.000118	0.000003435	0.0001111	0.0001249
4	[1,6,5,3,2,4]	0.0001193	0.000003454	0.0001124	0.0001262
4	[2,1,4,6,5,3]	0.0001198	0.000003461	0.0001129	0.0001267
4	[1,2,5,6,4,3]	0.0001198	0.000003461	0.0001129	0.0001267
4	[1,2,4,6,5,3]	0.0001199	0.000003462	0.000113	0.0001268
4	[1,2,3,6,4,5]	0.0001199	0.000003462	0.000113	0.0001268
4	[1,5,3,4,6,2]	0.0001203	0.000003468	0.0001134	0.0001272
4	[1,6,4,2,5,3]	0.0001209	0.000003477	0.0001139	0.0001279
4	[1,5,4,3,6,2]	0.000121	0.000003478	0.000114	0.000128

4	[1,2,6,4,3,5]	0.0001211	0.00000348	0.0001141	0.0001281
4	[1,4,6,2,5,3]	0.0001214	0.000003484	0.0001144	0.0001284
4	[1,2,4,3,6,5]	0.0001219	0.000003491	0.0001149	0.0001289
4	[1,2,4,6,3,5]	0.0001219	0.000003491	0.0001149	0.0001289
4	[1,5,6,3,2,4]	0.000123	0.000003507	0.000116	0.00013
4	[1,2,4,5,6,3]	0.0001245	0.000003528	0.0001174	0.0001316
4	[1,4,5,2,6,3]	0.0001252	0.000003538	0.0001181	0.0001323
4	[1,5,6,3,4,2]	0.0001266	0.000003558	0.0001195	0.0001337
5	[2,5,1,3,6,4]	0.000129	0.000003591	0.0001218	0.0001362
5	[1,2,3,4,6,5]	0.000129	0.000003591	0.0001218	0.0001362
5	[1,3,6,2,5,4]	0.000131	0.000003619	0.0001238	0.0001382
5	[1,3,2,4,6,5]	0.0001316	0.000003627	0.0001243	0.0001389
5	[2,1,5,3,6,4]	0.0001328	0.000003644	0.0001255	0.0001401
5	[2,3,1,5,6,4]	0.0001384	0.00000372	0.000131	0.0001458
6	[1,5,6,2,4,3]	0.0001423	0.000003772	0.0001348	0.0001498
6	[1,4,2,3,5,6]	0.0001436	0.000003789	0.000136	0.0001512
6	[1,5,6,2,3,4]	0.0001492	0.000003862	0.0001415	0.0001569
6	[2,3,5,1,6,4]	0.0001498	0.00000387	0.0001421	0.0001575
6	[1,5,3,2,6,4]	0.0001524	0.000003904	0.0001446	0.0001602
6	[1,5,2,3,6,4]	0.0001531	0.000003912	0.0001453	0.0001609
7	[1,3,5,4,6,2]	0.0001538	0.000003921	0.000146	0.0001616
7	[1,4,3,5,6,2]	0.000155	0.000003937	0.0001471	0.0001629
7	[1,5,2,4,6,3]	0.0001576	0.00000397	0.0001497	0.0001655
8	[1,2,5,3,6,4]	0.0001784	0.000004223	0.00017	0.0001868

Table 7 The full result of the triangular colour code with fixed bulk order (1,3,4,2,6,5) at $p_{idle} = 0.01p$ and $p = 3 \times 10^{-4}$.

Cluster	Boundary Gate Order	pL	sigma	95% CI Lower	95% CI Upper
1	[2,5,3,6,1,4]	0.0000595	0.000002439	0.00005462	0.00006438
1	[2,5,6,3,1,4]	0.0000637	0.000002524	0.00005865	0.00006875
2	[1,2,3,4,5,6]	0.0000925	0.000003041	0.00008642	0.00009858
2	[1,5,6,4,3,2]	0.0000927	0.000003045	0.00008661	0.00009879
2	[2,4,5,6,1,3]	0.0000942	0.000003069	0.00008806	0.0001003
2	[1,3,6,5,2,4]	0.0000944	0.000003072	0.00008826	0.0001005
2	[1,5,2,4,6,3]	0.0000946	0.000003076	0.00008845	0.0001008
2	[2,1,5,6,4,3]	0.0000949	0.00000308	0.00008874	0.0001011
2	[2,1,3,5,6,4]	0.0000953	0.000003087	0.00008913	0.0001015
2	[1,4,2,5,6,3]	0.0000957	0.000003093	0.00008951	0.0001019
2	[2,5,3,1,6,4]	0.0000962	0.000003101	0.00009	0.0001024
2	[1,3,5,6,4,2]	0.0000967	0.00000311	0.00009048	0.0001029

2	[2,4,6,5,1,3]	0.0000967	0.00000311	0.00009048	0.0001029
2	[2,1,5,6,3,4]	0.000097	0.000003114	0.00009077	0.0001032
2	[1,3,5,6,2,4]	0.0000981	0.000003132	0.00009184	0.0001044
2	[1,3,4,5,6,2]	0.0000987	0.000003142	0.00009242	0.000105
2	[1,2,4,6,3,5]	0.0000992	0.000003149	0.0000929	0.0001055
2	[1,5,3,6,2,4]	0.0000993	0.000003151	0.000093	0.0001056
2	[1,5,3,6,4,2]	0.0000995	0.000003154	0.00009319	0.0001058
2	[1,6,3,5,4,2]	0.0000996	0.000003156	0.00009329	0.0001059
2	[1,6,5,4,2,3]	0.0000997	0.000003157	0.00009339	0.000106
2	[2,3,5,1,6,4]	0.0000998	0.000003159	0.00009348	0.0001061
2	[1,2,5,3,4,6]	0.0001	0.000003162	0.00009368	0.0001063
2	[1,2,6,5,3,4]	0.0001002	0.000003165	0.00009387	0.0001065
2	[1,3,4,6,2,5]	0.0001003	0.000003167	0.00009397	0.0001066
2	[1,2,3,4,6,5]	0.0001005	0.00000317	0.00009416	0.0001068
2	[1,4,5,6,3,2]	0.0001007	0.000003173	0.00009435	0.000107
2	[1,6,3,5,2,4]	0.0001008	0.000003175	0.00009445	0.0001071
2	[1,5,6,2,4,3]	0.0001008	0.000003175	0.00009445	0.0001071
2	[1,6,2,4,5,3]	0.0001008	0.000003175	0.00009445	0.0001071
2	[1,6,5,2,4,3]	0.0001009	0.000003176	0.00009455	0.0001073
2	[2,1,4,6,5,3]	0.000101	0.000003178	0.00009464	0.0001074
3	[1,5,3,2,6,4]	0.0001015	0.000003186	0.00009513	0.0001079
3	[1,2,5,4,6,3]	0.0001016	0.000003187	0.00009523	0.000108
3	[1,6,4,5,2,3]	0.0001018	0.00000319	0.00009542	0.0001082
3	[2,5,1,3,6,4]	0.0001019	0.000003192	0.00009552	0.0001083
3	[1,2,3,6,4,5]	0.0001022	0.000003197	0.00009581	0.0001086
3	[2,1,4,5,6,3]	0.0001023	0.000003198	0.0000959	0.0001087
3	[2,1,3,6,5,4]	0.0001025	0.000003201	0.0000961	0.0001089
3	[1,4,3,5,6,2]	0.0001025	0.000003201	0.0000961	0.0001089
3	[1,5,6,2,3,4]	0.0001025	0.000003201	0.0000961	0.0001089
3	[1,5,4,6,3,2]	0.0001026	0.000003203	0.00009619	0.000109
3	[1,3,4,6,5,2]	0.000103	0.000003209	0.00009658	0.0001094
3	[1,6,5,2,3,4]	0.000103	0.000003209	0.00009658	0.0001094
3	[2,4,1,6,5,3]	0.0001032	0.000003212	0.00009678	0.0001096
3	[1,5,6,3,2,4]	0.0001032	0.000003212	0.00009678	0.0001096
3	[1,2,6,4,3,5]	0.0001033	0.000003214	0.00009687	0.0001097
3	[1,6,3,2,5,4]	0.0001034	0.000003215	0.00009697	0.0001098
3	[1,4,6,2,5,3]	0.0001035	0.000003217	0.00009707	0.0001099
3	[1,2,5,6,4,3]	0.0001036	0.000003219	0.00009716	0.00011
3	[1,3,5,2,6,4]	0.0001036	0.000003219	0.00009716	0.00011
3	[2,1,6,5,4,3]	0.0001038	0.000003222	0.00009736	0.0001102
3	[1,2,5,6,3,4]	0.0001038	0.000003222	0.00009736	0.0001102
3	[1,4,5,3,6,2]	0.000104	0.000003225	0.00009755	0.0001104
3	[1,4,5,2,6,3]	0.0001041	0.000003226	0.00009765	0.0001106

3	[2,4,6,1,5,3]	0.0001042	0.000003228	0.00009774	0.0001107
3	[1,3,5,4,6,2]	0.0001042	0.000003228	0.00009774	0.0001107
3	[1,4,3,2,6,5]	0.0001044	0.000003231	0.00009794	0.0001109
3	[1,4,6,5,2,3]	0.0001045	0.000003232	0.00009804	0.000111
3	[1,6,4,3,5,2]	0.0001045	0.000003232	0.00009804	0.000111
3	[1,6,4,2,5,3]	0.0001048	0.000003237	0.00009833	0.0001113
3	[1,2,6,5,4,3]	0.0001048	0.000003237	0.00009833	0.0001113
3	[1,2,6,3,5,4]	0.0001048	0.000003237	0.00009833	0.0001113
3	[1,4,5,6,2,3]	0.0001049	0.000003239	0.00009842	0.0001114
3	[1,5,4,2,6,3]	0.0001049	0.000003239	0.00009842	0.0001114
3	[1,6,4,5,3,2]	0.000105	0.00000324	0.00009852	0.0001115
3	[1,3,6,5,4,2]	0.0001051	0.000003242	0.00009862	0.0001116
3	[1,3,4,2,5,6]	0.0001051	0.000003242	0.00009862	0.0001116
3	[1,2,4,3,6,5]	0.0001055	0.000003248	0.000099	0.000112
3	[1,5,4,3,6,2]	0.0001055	0.000003248	0.000099	0.000112
3	[1,5,3,4,6,2]	0.0001055	0.000003248	0.000099	0.000112
3	[1,3,2,5,4,6]	0.0001057	0.000003251	0.0000992	0.0001122
3	[1,3,5,4,2,6]	0.0001058	0.000003253	0.00009929	0.0001123
3	[1,4,2,6,3,5]	0.0001058	0.000003253	0.00009929	0.0001123
3	[1,2,3,5,6,4]	0.0001058	0.000003253	0.00009929	0.0001123
3	[2,3,6,5,1,4]	0.0001061	0.000003257	0.00009959	0.0001126
3	[1,3,4,5,2,6]	0.0001062	0.000003259	0.00009968	0.0001127
3	[1,3,4,2,6,5]	0.0001063	0.00000326	0.00009978	0.0001128
3	[1,4,2,3,6,5]	0.0001065	0.000003263	0.00009997	0.000113
3	[2,3,1,5,6,4]	0.0001066	0.000003265	0.0001001	0.0001131
3	[1,6,3,4,5,2]	0.0001067	0.000003266	0.0001002	0.0001132
3	[2,1,6,4,5,3]	0.0001067	0.000003266	0.0001002	0.0001132
3	[1,5,6,3,4,2]	0.000107	0.000003271	0.0001005	0.0001135
3	[1,4,6,5,3,2]	0.000107	0.000003271	0.0001005	0.0001135
3	[1,3,6,4,5,2]	0.000107	0.000003271	0.0001005	0.0001135
3	[1,6,5,4,3,2]	0.0001075	0.000003279	0.0001009	0.0001141
3	[1,3,6,2,5,4]	0.0001076	0.00000328	0.000101	0.0001142
3	[1,6,5,3,4,2]	0.0001077	0.000003282	0.0001011	0.0001143
3	[1,6,5,3,2,4]	0.0001078	0.000003283	0.0001012	0.0001144
3	[1,3,2,5,6,4]	0.000108	0.000003286	0.0001014	0.0001146
3	[1,5,4,6,2,3]	0.000108	0.000003286	0.0001014	0.0001146
3	[2,1,6,5,3,4]	0.0001085	0.000003294	0.0001019	0.0001151
3	[2,6,1,4,5,3]	0.0001086	0.000003295	0.000102	0.0001152
3	[1,2,4,6,5,3]	0.0001095	0.000003309	0.0001029	0.0001161
3	[2,1,5,3,6,4]	0.0001096	0.00000331	0.000103	0.0001162
3	[1,2,4,5,6,3]	0.0001102	0.000003319	0.0001036	0.0001168
4	[1,2,6,4,5,3]	0.000111	0.000003331	0.0001043	0.0001177
4	[1,6,2,3,5,4]	0.0001115	0.000003339	0.0001048	0.0001182

4	[1,4,6,3,5,2]	0.0001115	0.000003339	0.0001048	0.0001182
4	[1,5,2,6,4,3]	0.0001145	0.000003384	0.0001077	0.0001213
4	[1,4,3,5,2,6]	0.000115	0.000003391	0.0001082	0.0001218
4	[1,2,3,6,5,4]	0.0001154	0.000003397	0.0001086	0.0001222
4	[1,4,3,2,5,6]	0.0001174	0.000003426	0.0001105	0.0001243
4	[1,2,3,5,4,6]	0.000118	0.000003435	0.0001111	0.0001249
4	[1,6,2,5,3,4]	0.0001183	0.000003439	0.0001114	0.0001252
4	[1,5,2,6,3,4]	0.0001183	0.000003439	0.0001114	0.0001252
4	[1,3,2,6,5,4]	0.0001187	0.000003445	0.0001118	0.0001256
4	[1,3,2,4,6,5]	0.000119	0.000003449	0.0001121	0.0001259
4	[1,4,2,3,5,6]	0.0001195	0.000003457	0.0001126	0.0001264
4	[1,6,2,5,4,3]	0.0001202	0.000003467	0.0001133	0.0001271
4	[1,4,2,6,5,3]	0.0001203	0.000003468	0.0001134	0.0001272
4	[2,3,5,6,1,4]	0.0001206	0.000003473	0.0001137	0.0001275
5	[1,5,6,4,2,3]	0.0001227	0.000003503	0.0001157	0.0001297
5	[1,3,2,4,5,6]	0.0001241	0.000003523	0.0001171	0.0001311
5	[1,2,4,5,3,6]	0.0001248	0.000003532	0.0001177	0.0001319
5	[1,4,3,6,5,2]	0.0001279	0.000003576	0.0001207	0.0001351
5	[1,2,4,3,5,6]	0.0001302	0.000003608	0.000123	0.0001374
6	[1,5,2,3,6,4]	0.0001445	0.000003801	0.0001369	0.0001521
6	[1,2,5,3,6,4]	0.0001456	0.000003815	0.000138	0.0001532
6	[2,6,4,1,5,3]	0.000153	0.000003911	0.0001452	0.0001608

